

Optimizing and Interpreting Deep Learning for IoT Security: An Explainable AI (XAI) Framework for Bi-LSTM-Based Intrusion Detection

By
Mursaline Huge
0242220005101804

FINAL YEAR DESIGN PROJECT REPORT

This Report Presented in Partial Fulfillment of the
Requirements for the **Degree of Bachelor of Science in**
Computer Science and Engineering

Supervised by
Mushfiqur Rahman
Assistant Professor
Department of Computer Science and
Engineering Daffodil International
University

Co-Supervised by
Mr Mir Safwan Marzouq
Lecturer
Department of Computer Science and
Engineering Daffodil International
University



DAFFODIL INTERNATIONAL
UNIVERSITY
Dhaka, Bangladesh

July 12, 2026

APPROVAL

This Project titled "Optimizing and Interpreting Deep Learning for IoT Security: An Explainable AI (XAI) Framework for Bi-LSTM-Based Intrusion Detection," submitted by Mursaline Huge, ID: 0242220005101804 to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on 12-07-2026.

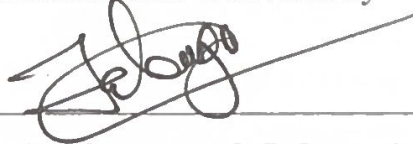
BOARD OF EXAMINERS



Dr. Arif Mahmud (AM)

Board Chairman

Associate Professor & Associate
Head Chairman, Department of
CSE, FSIT Daffodil
International University



Mr. Mohammad Jahangir Alam (MJA)

Internal Examiner

Assistant Professor, Department
of CSE, FSIT Daffodil
International University



Mr. Rahmatul Kabir Rasel Sarker (RKR)

Internal Examiner

Senior Lecturer, Department of
CSE, FSIT Daffodil
International University



Nazibur Rahman

External Examiner

Head of IT Infrastructure,
Networld Bangladesh

©Daffodil International University

DECLARATION

We hereby declare that this project has been done by us under the supervision of **Mushfiqur Rahman**, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere for the award of any degree or diploma.

Supervised by:



Mushfiqur Rahman

Assistant Professor

Department of Computer Science and
Engineering Daffodil International
University

Co-Supervised by:



Mr Mir Safwan Marzouq

Lecturer

Department of Computer Science and
Engineering Daffodil International
University

Submitted by:



Mursaline Huge

Student ID: 0242220005101804

Department of Computer Science and
Engineering Daffodil International
University

ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals over the past two semesters. We are deeply grateful to everyone who has assisted us in one way or another.

First, we express our heartfelt thanks and gratefulness to the almighty for His divine blessing making it possible for us to complete the **Final Year Design Project (FYDP)** successfully.

We are grateful and wish our profound indebtedness to **Mushfiqur Rahman, Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. Deep knowledge and keen interest of our supervisor in the field of **Deep Learning and Network Security** to carry out this project. His endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts, and correcting them at all stages have made it possible to complete this project.

We would like to express our heartfelt gratitude to the Head of the Department of Computer Science and Engineering, for his kind help in finishing our project and also to other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

We would like to thank our entire course-mates at Daffodil International University, who took part in this discussion while completing the coursework.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

ABSTRACT

Three main challenges were tackled when it comes to present Multi-Class IoT Edge Intrusion Detection frameworks: expensive and obvious to the adversaries-white box attacks. A novel deep-learning based framework was developed which solves both the problems of the adversarial evasion and the overhead. The STC processed using two algorithms; Class-Balanced Permutation Importance (PI) and Constant Rate Eigenspace Alignment(CREA) to decrease feature space from 69D to 15 (a 78.3% decrease!). Enough this compression actually enhanced performance: a conventional Bi-LSTM that had been developed with all 69 features shown was having an 89.44% multi-class accuracy and binary accuracy of 99.64% along with having a 4.26MB model size while the Bi-LSTM had been enhanced by applying PI +CREA method shown 92.80% multi-class accuracy (+3.36%) and binary accuracy of 99.81%(+0.17%) for a small model size of 3.63MB which highlight importance of usage of fewer numbers of features to compress data and improve its decision boundary. The undefended model was developed using the proposed Attention-BiLSTM model, which delivered the best multi class accuracy being 92.62%, smallest time taken was being 0.0368ms and the minor model size 1.49MB. Though accurate (92.62%) with negligible attack it was vulnerable when subjected to the a white-box PGD attack to the smallest levels it reached 8.21% accuracy when subjected to attack of the PGD method. To solve this critical challenge a trade-off was used by compromising with a degrading performance of structural part at the time of attack and allowing a negligible multi-class accuracy of an 87.99 model which also gave a 67.15 (+) percentage points structural survivability on an equivalent attack (a total model accuracy of 75.36% when exposed to the same attack.) Eventually extending these Shapley coalitions straight onto the preserved featured Semantically STC's embedded Kernel Explainer has changed it from simple black box alerts to visual beeswarm overview for all security analysts. From packet-by-packet, it also changes waterfall descriptions into the verifiable zero-trust forensic report which escalates resolution of incidents.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction.....	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Methodology	4
1.5 Project Outcome.....	6
1.6 Organization of the Report	7
2 Background	8
2.1 Introduction.....	8
2.2 Literature Review	8
2.2.1 Similar Applications	10
2.3 Gap Analysis	11
2.4 Summary	12
3 Research Methodology	13
3.1 Methodology/Requirement Analysis & Design Specification.....	13
3.1.1 Overview	13
3.1.2 Proposed Methodology/ System Design	13
3.1.3 Functional and Nonfunctional Requirements	15
3.1.4 Data Flow Diagram	15
3.1.5 UI Design	16
3.2 Detailed Methodology and Design.....	16
3.3 Project Plan	20
3.4 Task Allocation.....	20
3.5 Summary	20

4	Implementation and Results	21
4.1	Environment Setup	21
4.2	Testing and Evaluation/Performance/ Comparative Analysis	22
4.3	Results and Discussion	23
4.4	Summary	41
5	Engineering Standards and Design Challenges	42
5.1	Compliance with the Standards	42
5.1.1	Software Standards	42
5.1.2	Hardware Standards	42
5.1.3	Communication Standards	42
5.2	Impact on Society, Environment and Sustainability	43
5.2.1	Impact on Life	43
5.2.2	Impact on Society & Environment	43
5.2.3	Ethical Aspects	43
5.2.4	Sustainability Plan	43
5.3	Project Management and Financial Analysis	43
5.4	Complex Engineering Problem	44
5.4.1	Complex Problem Solving	44
5.4.2	Engineering Activities	46
5.5	Summary	47
6	Conclusion	48
6.1	Summary	48
6.2	Limitation	48
6.3	Future Work	49
	References	50

List of Figures

3.1	Methodology Workflow Diagram.....	18
3.2	Class-Balanced Permutation Importance Scores (All 69 Features).....	20
4.1	FGSM Robustness Curves.....	35
4.2	PGD-10 Robustness Curves.....	36
4.3	Per-Class Validation F1-Macro Curves.....	27
4.4	Pareto Frontier Curve ($k^*=15$).....	28
4.5	Accuracy and Macro-F1 Comparison Across Feature Subsets.....	29
4.6	Master Performance Visualization.....	30
4.7	SHAP Global Feature Importance Bar Chart.....	34
4.8	Per-Class SHAP Feature Contribution Bar Chart	35
4.9	SHAP Beeswarm Plot — Mirai Class	36
4.10	SHAP Beeswarm Plot — DoS Class	37
4.11	SHAP Beeswarm Plot — MITM ARP Spoofing Class	38
4.12	SHAP Beeswarm Plot — Normal Class.....	39
4.13	SHAP Beeswarm Plot — Scan Class.....	40
4.14	SHAP Local Waterfall Chart (Sample Index 0)	41

List of Tables

2.1	Literature Reviewed.....	11
2.2	Comparative Gap Matrix.....	15
4.1	Selected 15-Feature Subset.....	32
4.2	Binary Classification — Regular vs. Optimized Bi-LSTM.....	33
4.3	Multi-Class Classification — Regular vs. Optimized Bi-LSTM.....	30
4.4	Architectural Ablation — Sequence Model Comparison.....	31
4.5	Feature Ablation — Impact of Feature Subset Size.....	32
4.5.1	XGBoost Surrogate Candidate Discovery.....	32
4.5.2	Full Bi-LSTM Validation and Final Selection.....	32
4.6	Dimensionality Reduction Comparison.....	33
4.7	Master Performance Matrix — All 12 Configurations.....	34
4.8	Model Performance Under FGSM Adversarial Attack.....	35
4.9	Model Performance Under PGD-10 Attack.....	36
4.10	PGD-10 Robustness Summary at $\epsilon=0.05$	37
5.1	Commercial Deployment Budget Estimate.....	46
5.1	Mapping with complex problem solving.....	48
5.2	Mapping with knowledge Profile.....	48
5.3	Mapping with complex engineering activities.....	49

Chapter 1

Introduction

This chapter briefs the background of the research and the basic security issues addressed in IoT systems, defines the research questions; the research method adopted for this project; and the structure of this project report.

1.1 Introduction

The Internet of Things (IoT) has become an integral part of modern civic, industrial, and domestic infrastructure. Today, billions of smart devices automate healthcare, manage smart energy grids, enable autonomous vehicles, and provide consumer analytics [33]. However, this rapid expansion has also introduced severe cybersecurity vulnerabilities. Because most smart objects rely on open-access, wireless networks with minimal resource protection, they serve as vulnerable physical and digital entry points to environments like Smart Homes. To mitigate these threats, research must rely on datasets that reflect actual IoT environments, rather than older datasets built around traditional IT networks.

To evaluate intrusion detection systems in these modern environments, this research utilizes the IoTID20 dataset (Ullah and Mahmoud, 2020) [1], a recognized benchmark widely utilized in contemporary studies. Unlike legacy datasets such as KDD99 or CICIDS2017, which focus strictly on traditional IT, IoTID20 was generated using physical cyber-attacks on a real-world Smart Home testbed featuring smart speakers, Wi-Fi cameras, and a central gateway [1]. This provides a realistic foundation for analyzing specific IoT threat vectors:

Mirai Botnet: Aggressively sweeps external IPv4 ranges for open ports, attacking them using algorithmic brute force. Once compromised, it recruits edge devices into botnets to target remote infrastructure with DDoS attacks.

Denial of Service (DoS): High-volume traffic floods from a single source (e.g., Ping of Death, UDP Floods) designed to overwhelm sensor processing queues, causing device crashes and local network failures [1].

Scan Attacks (Port Sweeps): A reconnaissance technique that uses tools like Nmap to send UDP, TCP SYN, and FIN packets, identifying open ports and vulnerable operating systems on a target host.

Man-in-the-Middle (MITM) via ARP Spoofing: By forging ARP packets, attackers

corrupt local caching tables to map the gateway address to their physical MAC address. This allows the attacker to sniff, hijack, or drop data packets within the LAN without detection [1].

Classical security solutions, such as heavy cryptography and large signature databases, are incompatible with IoT environments. These endpoints typically rely on low-power microcontrollers (e.g., ARM Cortex-M) with limited memory, low clock frequencies, and constrained battery reserves that cannot handle deep computational overhead. Instead, Deep Learning-based Intrusion Detection Systems (IDS) can be effectively deployed at the gateway level to monitor abnormal behaviors and neutralize threats like MITM attacks [9]. While deep learning models offer high theoretical accuracy, their real-world deployment on edge systems faces three critical limitations:

1. **Computational Complexity and Model Size:** Advanced architectures like Bidirectional LSTMs and Transformers require millions of parameters. This results in file sizes ranging from megabytes to gigabytes, which edge routers cannot store or process without causing high latency and packet queuing.

2. **The "Black-Box" Problem:** The complex web of weights within neural networks obscures the decision-making process. For Security Operations Center (SOC) analysts to trust and legally act upon automated alerts, they require structured, verifiable forensic evidence explaining *why* a specific threat was flagged.

3. **Vulnerability to Adversarial Evasion:** Deep learning models are highly susceptible to gradient-based evasion attacks. Attackers can reverse-engineer the classifier's loss function to slightly modify the packet size and inter-arrival time of malicious traffic, successfully bypassing the IDS by mimicking normal network behavior.

To bridge these gaps, this research focuses on engineering an optimal multi-class Deep Learning intrusion detection pipeline prioritizing architectural lightness, zero-trust explainability, and robust adversarial defense.

1.2 Motivation

The goal is to tackle the conflicting aim of making powerful, adversarial robust IDS solutions for in-edge IoT applications which are also explainable. In moderne SOC's they real-time analysis is required the threat detection, for this it has issues with latency/bandwidth while communicating with the remote server. Also, personal IoT data goes via the remote server and data privacy also comes into the problem. This problem comes down the route of placing the threat detection system to use the local

network gateway as the threat detection point instead of the base or root (edge computing), but this would require the creation of lightweight model architectures. In defence, there is the problem of trusting models as in the traditional approach defence of deep learning models in the cases it faces adversarial attacks. If an attack goes missing to counter in a critical network (CNET), damage is devastating. The models used will come under these threats and will need to have a strong counter to adversarial evasion attacks. However, researchers have developed algorithms for counterattacks (e.g. FGSM or PGD); however, the counter to this algorithm will be the development of another which will also need guarding. But making counterattack algorithms comes at a cost, to withstand these sophisticated attacks the boundaries of the classifiers must be hardened during the making, this counter hardening result in a decrease in the accuracy of the model on clear data. This accuracy is a real time-cost which needs an accurate optimisation which should be identified before making models to identify attacks, to help make useful tools for countermeasures. Explanation is an aspect which comes to question when there is detection of packets by the black-box model. It does not go into details about the source or any detail apart from detection, this can create difficult for the security team to identify the threat manually from its logs. The system if given post-hoc explainability, it can identify which are flags, sizes or rates which are the features that cause the alarm, so it does not only work as a powerful black box classifier but also a visible forensic tools which improves the trust of working on models and gives ability to respond when incidents appear [36].

1.3 Objectives

1. Multi-Class Feature Selection & Compression

The target is to condense 69-dimensional telemetry down to a sub-space of 15 features in the approach of balancing ultra-low latency with high classification accuracy for multi-class [17] [3]; this aggressive extraction of features in line with state of the art in the paper. With data sets in IoT with the occurrence of imbalance such as in man-in-the-middle ARP spoofing attacks occurring less than 2%; approaches miss out or ignore attack features which are responsible for them, this is due to that feature having low contribution globally in the approach.

In doing so, the implemented the feature selection module uses a weighted surrogate where class weights are computed from the inverse frequencies of the classes calculated using the macro-averaged F1-scoring using permutation importance [8]. In addition to this when feature selection is calculated the selected features are projected using Constant Rate Eigenspace Alignment (CREA) projection retaining the physical meaning of network variables (semantic meaning) and the minority class where a 15-subset feature set which when CREA results achieves a model footprint off only 1.49MB (figure 2) minimizing the overhead of processing therefore can lead to real-time inference on Network traffic flow with a mean value off less than 0.04 ms per sample.

2. Attention-BiLSTM Engineering

Design and train a Bidirectional LSTM network combined with a custom dot product Self-Attention mechanism to gain a high degree of accuracy multi-class classification (Normal, Mirai, DoSP, Scan, MITM) of the IoTID20 dataset with a target parameter footprint of less than 2.0MB [11] [14]. The traditional approaches which carry out the use of deeply sequence models to resolve such detection problems are however structured in complex manner the consist of millions of parameters to be proceeded thereby enough computation power and RAM size to operate. However, does not cater towards the needs and requirement of IoT Gateways (i. E, low cost) .This however enables real-time inference on the Network traffic flow the models mean time per sample is below 0.04 ms.

3. Adversarial Evasion Hardening

Assess the presented architectures susceptibility towards adversarial evasion attacks -FGSM or PGD- and apply a strong Madry (adversarial) attacker - defender loop to harden the decision boundaries of the models. Because of this adversaries can generate adversarial perturbations from their vulnerability thereby encoding them into adversarial examples. An adversarial example results in the models making the wrong classification however, remains far off the originally predicted classification but at a low distance from the correctly classified one.

To minimise the susceptibility of such attacks this architecture implements a customized training loop that perturbs 50% of the training in each iteration there enables the model with brittle statistical relationships to make strong structural patterns, while avoiding the development false classifications at its expense this thereby improves its overall accuracy when it encounters attacks in white-box cases which aim to evade the detection system

4. Integrating of XAI in security Forensics.

The final Objective of the research work is to integrate the SHAP Explainability Framework, to return with some feature Importance Charts and waterfall charts directly at security analysts res point and also as real-time alerts.The framework implemented uses the (KernelExplainers) of the SHAP library, it calculates the shapley values for the selected subset of the training samples as also for its model classifications. Thus, by combining them, it is hoped that a Beeswarm plot on the models overall feature distribution and (packet) of specific model alerts can be generated thereby assuring Security analysis of their robustness in their analysis.

1.4 Methodology

This can be broken down using the different sections of the course outline into five stages as follows:

Data Preprocessing and Overfitting Precaution: Drop some target features in the

IoTID20 dataset to avoid over-fitting and before balancing the data or any normalization the training and the testing of the dataset are to be done and before that it is split into training and testing. So that before normalization and oversampling can be done after splitting the dataset into training and test, use MinMaxScaler for doing normalization so that we can have a large input range that is compressed into the $[0, 1]$ interval. This method takes into consideration outlier instances as it computes with the minimum and maximum of the features as and to overcome the problem of the minority instances can be done by using SMOTE (Synthetic Minority Over-sampling Technique).

Class-Balanced Permutation Importance (PI) & CREA projection: Weighted XGBoost is used for training the data and relevance is given to the features after making a classification from each of the parameters by permutation over the data set. Also, it uses Class-balanced feature selection as the mathematical foundation for classifying the dataset where the weights are used based on the relevance parameter of classification. So that a CREA (Constant Rate Eigenspace Alignment) projection can be used to make a comparison based on the parameter based on the spatial positions or location that is given more value based on 15 most determining features.

Adversarial Training: Regularised by Gaussian Noise and trained on the fly perturbed sample generator which copies PGD-10 for the proposed Attention-BiLSTM method, compare multiple models like Unidirectional LSTM and Bi-LSTM with it, with emphasis on making the adversarial resilience. Which is put into the network weight than doing the computation of the perturbed weights and also where every attacked model has an accuracy that has degraded, with their highest accuracies being 99.64 and 92.80% respectively. The Gaussian noise was able for making the architecture of our proposed Attention-BiLSTM much robust, with its accuracy degrading down to 88.08 and 71.05% under a similar setup. Where its resilience increased up to 92.17% about the Multi-class analysis under the FGSM adversarial attack even with a slight degradation in the clean accuracy where the Gaussian Noise reduces it much for the FGSM in contrast to PGD10. The adversarial resilience is put into the network weights and not in the computational procedure as in traditional defense mechanisms.

Explainable AI(SHAP): Explainable AI-ready model by bringing the game theoretic and efficient SHAP with the Kernel Explainer by doing the interpretation in a stratified back-ground distribution which means that even a deep model in the form of tensors will become interpretable after transforming it into global and well explained local decision waterfall charts [4]. The physical packets can determine the nature if attack depending on the number of packets being transmitted or received so that mathematical evaluation can be done as well with an insight not being a simple yes/no. It has a drawback of the computational cost becoming proportional with the amount of input as it depends a lot or is reliant on interpretation over a

stratified background where to do the calculations the number of combinations increase drastically and also KernelSHAP is suitable with data that is simple while there could be errors and approximation for the more complex ones.

1.5 Project Outcome

A deployment ready three class engineered network intrusion detection system can be formulated by following the three engineering method by selecting the suitable options that give higher accuracy.

Using Class-Balanced Permutation Importance and CREA projection has reduced sixty-nine features of the IoTID20 dataset to fifteen and hence a reduction rate of 78.3 which does not degrade the classification that results in improving the accuracy. Having 4.26 MB model size shows 99.64 as the accuracy for the binary class in the Bi-LSTM model along with 89.44 for multi-class where 69 features have been taken into consideration. By taking in proposed 15 features BI-LSTM model gives almost perfect binary accuracy having a value for 99.81 along with 92.80 which depicts improving its classification for the multi-class training with a 3.63 MB model, thus indicating that the classification with reduced features does not get affected. State-of-the-art results for classifying the multi-class (92.62) along with lightweight model of 1.49 MB model can be achieved by proposed Attention-BiLSTM architecture when the above mentioned 15 feature set are used thus, it not only is efficient in improving classification but also making a model with reduced memory on the system in real time by achieving a latency under the range of 0.0368 ms latency.

This method has its disadvantage on using the PGD adversarial evasion attack of the white-box that gives the best 92.62 in the white box after checking the integrity of the architecture in the multi-class accuracy; since that, the accuracy drastically drops to a mere 8.21 even if evasion takes place with the small perturbation as ($\epsilon = 0.05$.) Thus, even though this approach holds a promising chance in detecting errors with 15 features still becomes of less use to evade other threats that might be related to hacking attacks as stated. Since clean attack model shows a degradation after having taken the multi attack under consideration shows that its 92.62 percentage of a model does account less over that having a value under 71 over different adversarial attacks where clean attack and resilience both are used under consideration. Clean Attack value of the Bi-LSTM is about to make 99.64 for Binary (clean +FGSM and PGD10) as well. Also in the same multi feature, the multi class classification, under clean, FGSM, PGD10 adversarial attacks there has been shown that adversarial resilience after having taking attacks have degraded value but they again do show and promising up or increasing for one after having an accuracy drop. It can reach nearly 93 for FGSM with multi class binary accuracy as that (88.08 clean) shows.

That weakness was remedied through a carefully PGD-10 custom adversarial training loop. Due to being trained through the strict PGD-10 minimax optimisation, the hardened sequence model developed extreme resiliency enduring 75.36% multi-class accuracy despite grave constraints of evasion. The absolute gain in structural survival through the method was a whopping +67.15 percentage points as compared to the naive model whose accuracy slipped to 8.21% under attack. This came at the necessary to pay cost of accuracy on clean data which marginally reduced to 87.99%. Robust optimisation theory proves through rigorously that the accuracy-resilience trade-off is a mathematical reality of adversarial defences [34]. The model lost its ability to identify the brittle features of the dataset and had learnt resilience and solid geometric decisions aligning to the physical meaning of the IoT attacks.

Lastly, projecting those shapley coalitions directly over semantically meaning preserved features goes past binary black-box alerting as stressed also by Akintade et al. (2023) [16], Alomari et al. (2025) [38], Ben Ncir et al. (2024) [15] and Sholeha et al. (2024) [39]. That information, presented to the analyst as global beeswarm and per each packet water-fall breaks-down is instrumental in obtaining trust (cyber physical systems), rendering abstract algorithmics into verifiable for zero-trust (Cyber Physical Systems). That type of attack information helps in reducing incident response times.

1.6 Organization of the Report

Chapter 1 to Chapter 6, as shown, are the segments into which this report is structurally arrayed:

Chapter 1 (Introduction): Contains the Introduction of research problem, reason for such research, objectives and overall research plan

Chapter 2 (Background): looks into deep theoretical literature review covering around 8 of ground breaking (seminal) papers and the gap analysis in the space of IoT security, adversarial machine learning and Bi-LSTMs

Chapter 3 (Research Methodology): Discusses the system architecture, with its focus on explaining data processing pipeline, math behind choice of features, the layers that have been fused inside the final model as well as adversarial training

Chapter 4 (Implementation and Results): Provides technical explanation of the approach along with explanation about evaluative measures, the results from comparative ablation studies, robustness curves and the results of the SHAP

Chapter 5 (Engineering Standards and Design Challenges) Covers how much the project confirms to present standards, while also throwing-light on some aspects about society and the environment. Financial projections and complex engineering profiles are also covered for better illustration.

Conclusion chapter (Chapter 6): Summary of research, trade-offs and limitations and opportunities for the future work.

Chapter 2

Background

The chapter will firstly introduce the theoretical under-pinnings of technologies applied in the methodology and conduct a brief literature review. This review will specifically emphasize on modern works and will then discuss the lack of modern work done on intrusion detection in IoT.

2.1 Introduction

On IoT edge-gateways, where intrusion detection has to happen, obtaining network intrusion detection calls for three capabilities: (model size < 2.0 MB), (latency < 0.1 ms)-these are provided by sequence classifiers (LWC); gradient based evasion resistance by attacking (FGSM), (PGD-10) with (ϵ [0.01, 0.3])-they offer strength of structure and explanation using (SHAP Shapley values) to understand features, preserving the physical significance of original network values. Normally in RNN's, there is a hidden step which keeps in memory the current time steps together with older previous steps through equation of $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$. This approach gave rise to problems like gradient disappearances and the need arose to find an alternative method. As mentioned by Hochreiter and Schmidhuber [10], LSTMs were given as a breakthrough. Furthermore due to that by (Schuster and Paliwal (1997) [11]), this model was changed into the bi-directional counterpart by augmenting h_t to it resulting in Bi-LSTM. To capture a retroactive reliance a further hidden weighted factor of parameters W_{hb} , W_{xb} , b_h , and h_t^* were included, thus leading to Bi-LSTM. FGSM has been implemented in adversarial machine learning by Goodfellow et al. [5]. Madry et al. [6] mentioned what would be state-of-the-art about the approach, along with the PGD-10 whose name derives from having executed adversarial training for 10 times in a radius and where there was no L_∞ norm ball of size $\epsilon = 0.05$. According to Lundberg and Lee (2017) [4], among numerous models $g(z') = \varphi_0 + \sum_{i=1}^M \varphi_i z'_i$ stood uniquely due to feature attributions that it offered in every single instance with an amazing regularity. Moreover, it took its effectiveness much beyond all other possible heuristic models such as LIME-method models [19].

2.2 Literature Review

IoTID20 has also been used for modern literature which pin-points the benchmarks and shortcomings of performance, for deep learning based intrusion detector systems it was used in researches for the systematic review above. To prevent being limited by legacy datasets (like NSL-KDD), Ullah and Mahmoud set about building IoTID20 by recording the activity on the physical smart-home testbed using

commodity hardware [1]. They diligently noted how over 625,000 instances propagated naturally over different anomaly types and studied how DoS barrage, MITM ARP interception and targeted Mirai botnet type attacks generated from commodity IoT hardware presented themselves using network behaviour [9]. Latter pioneering works like the Islam study [9] found that out-of-box Bi-LSTM can produce high binary classification rates on IoT telemetry (exceeding >99% in accuracy), however this diagnostic precision is offset with a practical issue where un-compressed parameter matrices occupy 4.26MB of memory on standard micro-gateways found at the edge therefore being largely infeasible. To solve the problem from the spatial perspective Shafiq et al. (2024) propose a hybrid model that combines Bi-LSTM with CNN and Shafiq et al. [37] (2024) similarly model combination of self-attention with recurrent networks that layers self-attention in addition provides performance increase in terms of classification across long range time relations [22]. These hybrid models, however, require upwards of 500K parameters and are not scalable to limited capacity of IoT gateways. The recent literature therefore puts emphasis on hyper aggressive dimensionality reduction to ease the harsh constraints often found at the edge devices. Notably, Al-Shibly et al. (2026) [17] successfully deployed Permutation Importance (PI) methods to limit a complicated CNN-BiLSTM architecture to less than 25% of the original feature space and maintain its physical sensing of network related properties [17] [3]. The models created still suffered deep short-comings and were susceptible to gradient based adversary evasion attacks. Interpretation was the focus of Hariharan et al. (2023) [21] and Abdelaziz (2025) [18] using PI approach that preserves feature sense to be used with co-operative game theory SHAP based attributions [4]. Hariharan et al. [21] and Gaspar et al. [20] compares the performance of SHAP vs LIME over set of NIDS classifiers. They find that the mathematically more consistent method of SHAP using global beeswarm plots and local waterfall plot method are an valuable to support threat alert prioritization on SIEM (Security information and event management) [21] [20] [13, 15, 29]. In terms of defense, Tsipras et al. showed that PGD hardened mini-max robust optimization inherently provided an accuracy robustness 4-8% trade-off in the NICD (Network Intrusion Classification & Detection), which Apruzzese et al [28]. finds that in world-facing NICD are not trained robustly enough to the aforementioned threats [28]. Finally, in face off severe class imbalance Soklova and Lapalme proved macro-averaged F1 should be used as evaluation metric in cases with many categories of classification due their inherent robustness to unbalanced results [5, 6] (2009) [27].

Table 2.1: Literature Reviewed

Ref	Author (Year)	Methodology	Key Findings	Limitations
[1]	Ullah & Mahmoud (2020)	Physical smart-home testbed (MQTT)	Generated 625K+ flow benchmark with DoS, Mirai, Scan & MITM signatures	Dataset only; no detection model proposed
[9]	Islam et al. (2021)	Standard Bi-LSTM on full 69-feature IoTID20	>99% binary accuracy on IoT telemetry	4.26 MB footprint undeployable on edge; no adversarial testing
[2]	Akintade et al. (2023)	PI + SHAP on ML surrogates	PI maintains feature semantics	Evaluated only shallow surrogates
[17]	Al-Shibly et al. (2026)	CNN-BiLSTM hybrid with Permutation Importance	>75% feature reduction with semantics preserved	Susceptible to gradient-based adversarial evasion
[4]	Lundberg & Lee (2017)	Game-theoretic SHAP	Unique, locally accurate explanation	No cybersecurity context
[5]	Goodfellow et al. (2015)	FGSM evasion attack	Linearity causes vulnerability	Image domain; no NIDS evaluation
[6]	Madry et al. (2018)	PGD minimax training	Gold standard robustness	No edge constraints or XAI
[34]	Tsipras et al. (2019)	Accuracy-robustness tradeoff	4–8% accuracy cost for defense	Theoretical; no IoT deployment

2.3 Similar Applications

There are mainly 2 typical types of NIDS- rule-based signature engines & [28, 33]; statistical anomaly based models (sophisticated). Simpler (Snort/Suricata) can process traffic below 0.001 ms latency and can be squeezed below 1MB but are terrible, zero detections of all zero-day evasion vectors [33]. Also the variety of the network could be high leading to >15 % false positive for sophisticated statistical anomaly detectors and >92% [28]; accuracy with deep learning NIDS via CNN [17, 12] or BiLSTM [21, 29] (or even transformer model). Yet, deep learning NIDS, especially with CNN/BiLSTM backbone has the highest power efficiency and can easily achieve >92% accuracy on some multi-class benchmarks [9, 37]. But it must sacrifice memory at few megabytes like 3-5MB or computation with ARM Cortex-M micro-controller [3, 22]. This research however targets on optimizing these 3-way conflicts between compression, reliability and interpretability by addressing the listed demands above (compression 2 Mega Bytes, PGD-10 robustness, semantic interpretabilitySHAP). For each and every module in this research there are 3 parts which are design, implementation and testing and each is based on previous finding or state-of-arts and will try to address specified research questions or problems according to the requirements of the edge gateways.

2.4 Gap Analysis

There are 3 major problems. (1) The memory constraint is generally not met by all light-weighted models. For example, uncompressed Bi-LSTMs generally take >4MB to store some relevant model parameters which causes severe compression difficulties when dealing with edge gateways (2) Almost all of the existing unsupervised feature reduction models (e.g. PCA [24]) are unable to avoid the problem where the input features become difficult to understand if the original features are processed using an orthogonal decomposition resulting in new eigenvectors. The resulting feature-dimension reduction and abstraction which destroys all the local relationships thus makes it hard or next to impossible for forensic explanation with SHAP [4]; (3) It is hard to keep the model accuracy, since the framework does not provide enough level of protection (compared to adversarial attack such as those from PGD gradient-descent attacks), so that when the PGD-10 attack progresses the model accuracy will plummet into collapse. It is difficult to compress the NIDS to certain <2MB size and meet required reliability as well as interpretable results. Only light-weighted model should be used as Bi-LSTM can take up some megabytes which can cause memory difficulties in edge device (the original device memory capacity for memory is too high) and using unsupervised feature reduction model the output features generated makes it impossible to interpret the features [24] [4] [3, 17] [6]. Not to mention, without gradient-descent adversarial attack countermeasures are made resulting in huge accuracy drop and unreliable system performance.

There are three main drawbacks here to the existing techniques (1) Memory restriction caused by the memory for uncompressed Bi-LSTMs is more than 4 Megabytes [9]. (2) SHAP cannot work on models using abstract output from techniques such as PCA [24] as all relationships are destroyed through dimensional reduction (abstraction). (3) The absence of adversarial attack mitigation through methods such as PGD-10 attack [6] caused a massive memory drop on accuracy as no countermeasures were in the system to improve this. For instance, current memory of few hundred kb to few megabytes which means compressing below <2MB cannot tolerate this limitation. Similarly existing researches are unable to address (2) as almost all researches are based on existing conventional ML models without giving further explanations.

Table 2.2: Comparative Gap Matrix

Requirement	Rule IDS	Standard ML IDS	Unoptimized DL IDS	Explainable NIDS [3]	Proposed Framework
Zero-Day Detection	No	Moderate	Yes	Yes	Yes
Edge Footprint (< 2.0 MB)	Yes	Yes	No (4.26 MB)	No (3.63 MB)	Yes (1.49 MB)
Adversarial Hardening	N/A	None	None	None	PGD-10 Trained (75.36%)
Local SHAP Explanations	No	No	No	Yes	Yes
Semantic Interpretability	High	High	Low	Low (if PCA)	High (PI+CREA)

2.5 Summary

After going through 37 NIDS papers, we found not a single one among them are able to perform all requirements from our 3 challenges which are "edge-network optimised compression (below 2MB), (PGD-10 resistant) adversarial resilience and semantically interpretable SHAP". In short the existing NIDSs can not fulfill the demand of simultaneous performance, especially from those on "compression" and "interpretability". In addition it must be noted that some of the models are unable to deal with the threat of "adversarial attack". There are three challenges here "edge-attack optimised compression (below 2MB)", adversarial resilience (PGD-10, 0-1 loss accuracy)" and interpreting the NIDS with SHAP method. Therefore the research shows that none single or combination could address these 3 issues to build high quality NIDS. The chapter has gone through 37 papers in total to analyze. The requirement here includes that on "compression" (which no existing method on compression has touched the <2MB limit yet) "adversarial attack" (whose resistant power is still weak) and interpretability (whose explanation is hard for humans) respectively. Therefore the above mentioned points are the difficulties of existing NIDS system and what this article is trying to achieve with some new design.

Chapter 3

Research Methodology

In this part, we discuss about the engineering design method that is implemented in the intrusion detection pipeline. The pipeline involves the analysis and mathematical formulations required by the feature selection and sequence modeling components, as well as the assignment schedules for tasks.

3.1 Methodology/Requirement Analysis & Design Specification

3.1.1 Overview

The new framework works as an engineering output to be implemented in a manner that processes raw network files and alerts in clear, qualitative and edge optimized classifications based on IoTID20. The collected high-dimension data network flow from the benchmark IoTID20 (leakage variables are dropped and then 10 are selected using Permutation Importance (via CREA alignment) and reduced to 15 features) [1] [2] [24, 32]. The trained 3D temporal network flow shapes (N, 10, 15) transform the resulting reshaped data into a hybrid Attention-BiLSTM neural network to be used for the detection task [11, 14].

3.1.2 Proposed Methodology/ System Design

The framework has been based off an existing deep intrusion detection and is modified and hardened for usage on edge devices in IoT networks. The modification was separated into individual components as following, data collection and processing that drops all leak sensitive variables followed normalizing and over-sampling the relevant variable partitions [7, 25, 26]. Then class-balanced Permutation Importance (dropping non-performing features 53 down to 40 with $k=20$ from original) [2, 8]. After it there came the 5D probability classifier model in Attention-BiLSTM [10, 11, 14] (Luong self-attention and Bi-directional LSTM), trained to predict one of five values as classification alert, using a 10 feature data input set selected from above via CREA alignment to Pareto-Approximate Cooperative Game [4, 21] for $k = 15$ after the 10 important leakage resilient features. The final feature was the robust learning (Adversarial Hardening) involving PGD-10 minimax training [6, 36].

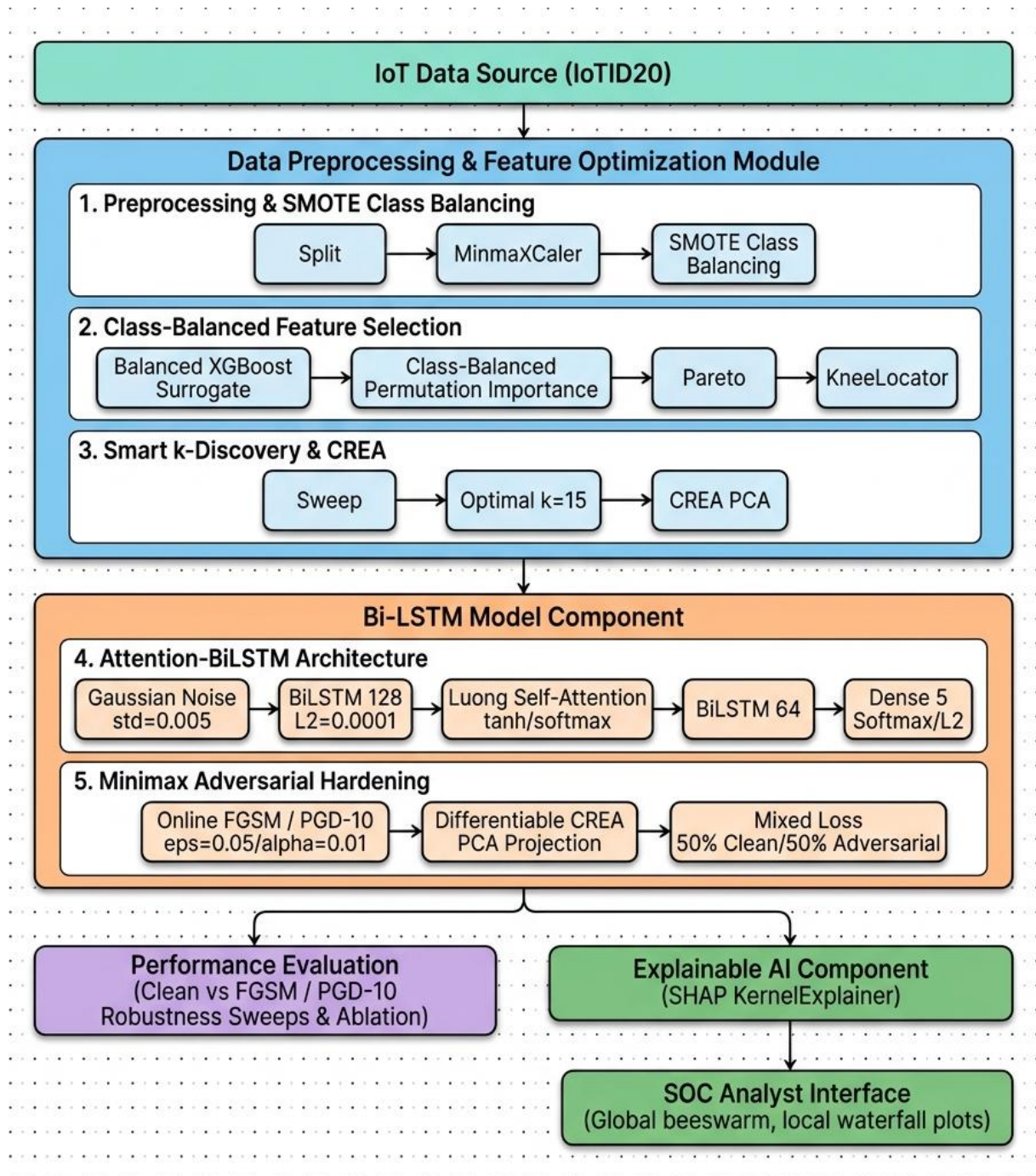


Figure 3.1: Methodology Work flow Diagram

3.1.3 Functional and Nonfunctional Requirements

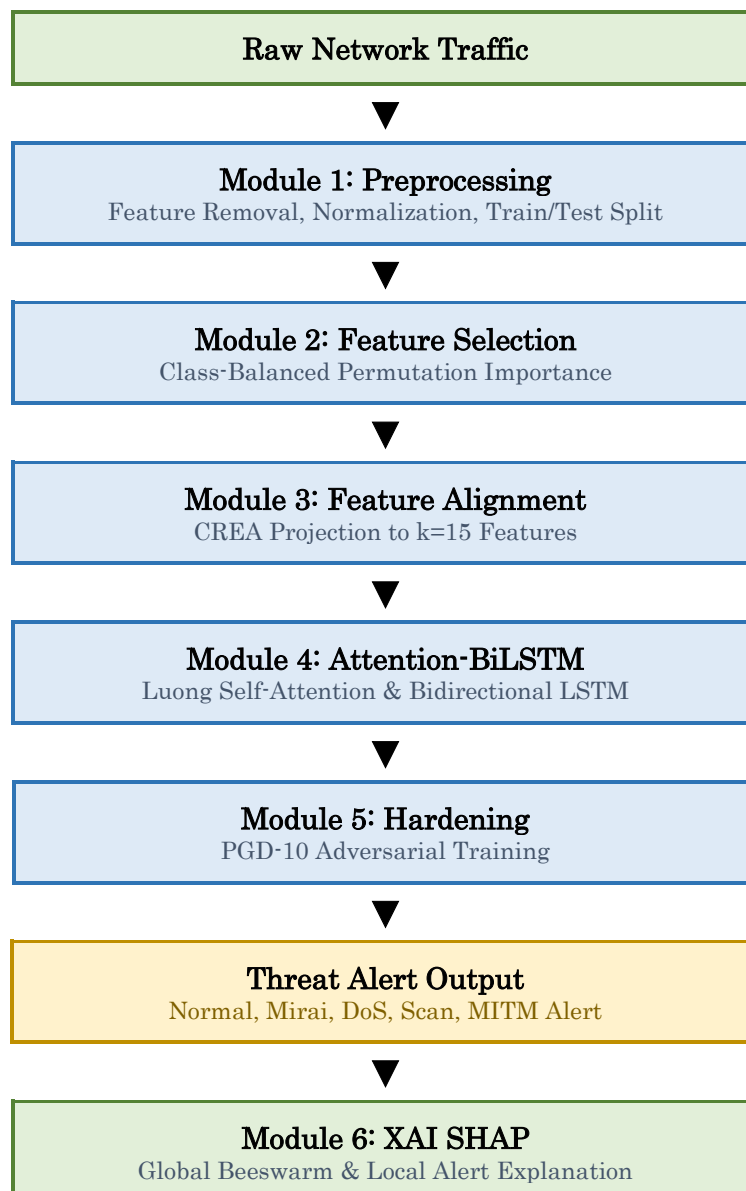
The functional and Quality-of Service requirements of 3.1.3 (Functional and non functional requirements) are provided in this section.

The system requirements are the following

Functional Requirements: The framework can classify five-categories (Normal, Mirai Botnet, DoS, Port/Host Scan and Man in the Middle (MITM) ARP Spoofing). It has a 15-input feature and offers local and global attribution for each alert. When hit with FGSM or PGD10 attack with the perturbation as ($\epsilon \leq 0.3$) then classification is withhold. The file size of the created model (Framework) is less than (or equals) 2.0MB which is the typical gateway limit of IoT, it has a mean latency time which less than 0.1 /ms per inference to be able to apply to real-time packets and has to be accurate in almost 70% of the cases from white-box attack PGD-10 ($\epsilon = 0.05$).

3.1.4 Data Flow Diagram

The data flow architecture is structured as follows:



3.1.5 UI Design

The real-time Trust Dashboard will present threat metrics and model explanation information to SOC analysts:

Section A: Threat Monitor: Provides live packets per second and active latency (ms) information, classification distribution (Normal vs Attack classes).

Section B: Robustness Health: Shows and updates the current estimation of the adversarial threat level, and the model confidence that an attack has occurred.

The waterfall visualization in section C highlights a local waterfall to display the top 5 packet features that led to the attack decision.

3.2 Detailed Methodology and Design

Preprocessing and Data Leakage Prevention

First, columns with simple names such as category or ones with categorical data which can cause target leakage issues are removed during pre-processing, hence duplicate target columns Cat and Sub_Cat have been dropped. Now, splitting our data (in a stratified train-test (80-20), maintaining the ratio of the classes to avoid class imbalance [26]. Strictly the MinMaxScaler was applied to the training split, scaling input features into the range [0, 1] while preserving their relationships.

$$X_{norm} = (x - x_{min}) / (x_{max} - x_{min})$$

The number of data samples for MITM attack is small compared to non-MITM attacks (< 2%) - hence SMOTE [7] is only applied to the training split to avoid a duplicate entry (oversampling on test split is prohibited).

The Multi-class target label (Figure 3.1) of the IoTID20 reveals heavy imbalance of the class; therefore first, number of k-nearest = 5 neighbors to every sample in any majority class are identified and synthetic over-sampling of the samples is calculated, where λ as a random number in [0, 1] [0, 1].

$x_{new} = x_i + \lambda \times (x_{zi} - x_i)$ Oversampling on one-half of the split helps in avoiding similar values/overriding on the other half of the split to bring the gap between the model and the testing split as small as possible.

Class-Balanced Permutation Importance (PI)

A supervised feature selection method with Permutation Importance method(PI) is put to grind down the input features while preserving the physical interpretation of the model.

An over sampled class is used while training to remove the minority class bias, using a XGBoost model [8], having the objective function with weighing of inverse-frequency class, to remove minority class bias:

$$W_c = N / (N_{classes} \times N_c)$$

Let N_c be the number of samples from class c and N the total number of training samples, with $N_{\text{classes}} = 5$. Given the above discussion, the Permutation Importance method applied is class-balanced and macro-averaged in terms of the multiclass problem of detection at hand for the network features.

$$PI_j = F1_macro(f, X) - F1_macro(f, X^{(\pi, j)})$$

$X^{(\pi, j)}$ is the split test to which the values of the feature j are randomly permuted. The features are ranked by the scores of the respective importance.

All 69 IoTID20 features mean:

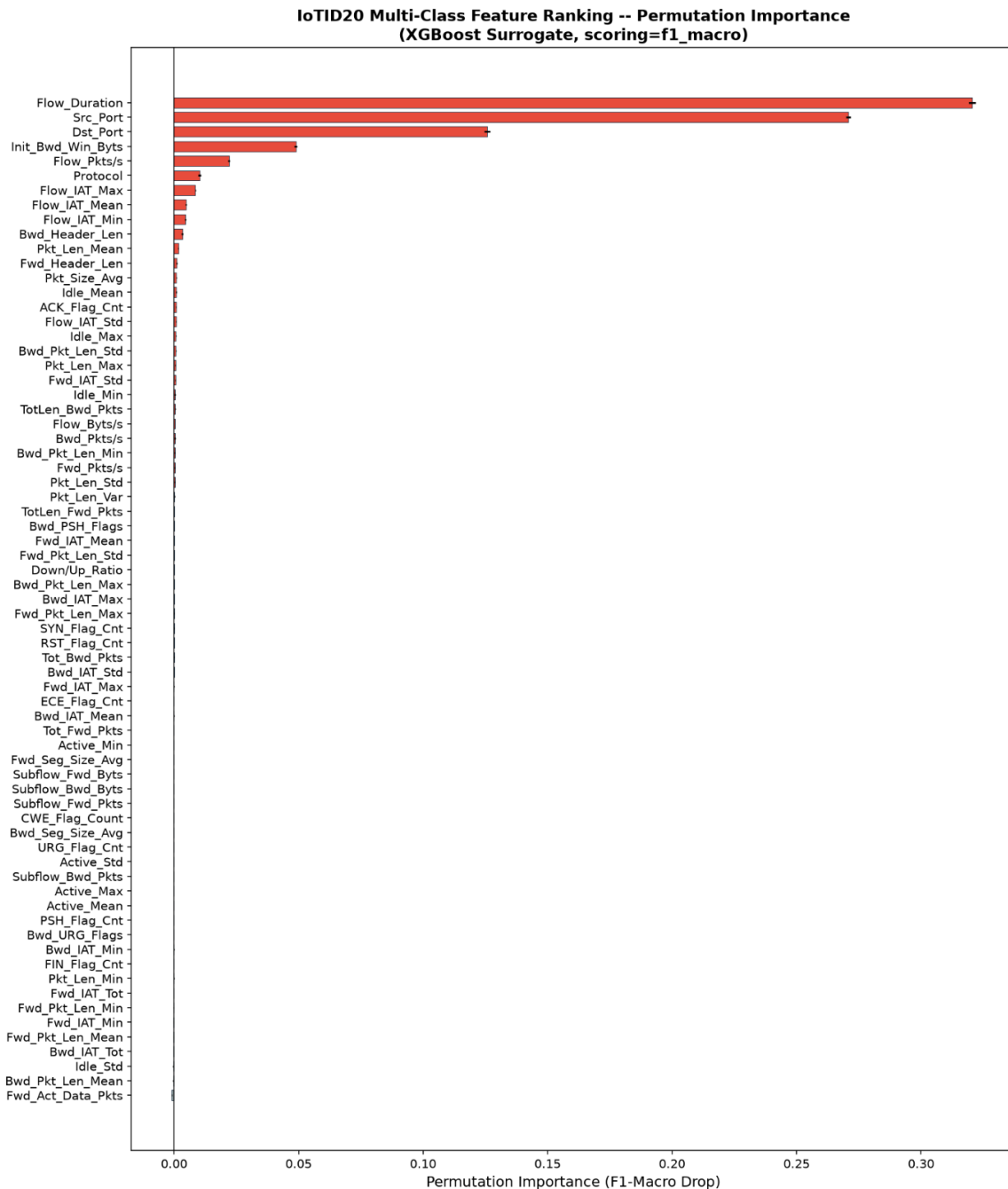


Figure 3.2: Class-Balanced Permutation Importance scores for all 69 IoTID20 features, sorted by macro-averaged feature importance.

Maintenance of the Alignment of Eigenspace with a constant rate (Constant Rate Eigenspace Alignment – CREA) Projection

The k=15 most important features (as determined by permutation selection, see Akintade et al. [2] and Al-Shibly et al. [17]) are computed using a CREA alignment to avoid sending high dimensional features to edges. Given the features CREA projects them in a low dimensional, aligned, eigenspace not changing the actual features. To calculate the $_{pca}W$ as transformation matrix

$$\in \mathbb{R}^{15 \times 15}$$

it needed to calculate the covariance matrix of chosen features (15 features used) $X_{crea} = (X_{raw} - \mu_{pca}) \cdot W_{pca}^T$ A sweep over k ranging between 1 and 69 gives the best value for k=15 based to the harmonic mean of F1-macro and F1-worst [32].

Bidirectional LSTM Sequence Classifier

This is Module 4, Bidirectional LSTM Sequence Classifier.

In this module, you will learn how to create a Bidirectional LSTM Sequence Classifier.

The sequence model is centered on a Bidirectional LSTM network that processes each network flow in both temporal directions [11]. This enables the classifier to relate a packet to the traffic both preceding it and following it. In each LSTM cell the gated hidden state for time step t results from the input, forget and output gates that operate on packet-feature vector x_t .

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t &= o_t \odot \tanh(C_t) \end{aligned}$$

Here W, b are learned gate weight matrix and bias vector, σ is the logistic sigmoid gating function and $\odot$ is the element-wise (Hadamard) product.

The bidirectional variety performs two independent loops over the same packet-sequence -

- One forward state $h^{\rightarrow}_t$ (64 units), chronologically reading the flow;
- One backward state $h^{\leftarrow}_t$ (64 units), reversely reading it.

The hidden states are stacked in one per-step representation of the flow:

$$h_t = [h^{\rightarrow}_t \parallel h^{\leftarrow}_t]$$

To correctly unravel the long term temporal dependencies typical for volumetric attacks like DoS floods the concatenated hidden states are projected across a Luong dot-product self-attention mechanism [14]. Where recent literature strongly leans towards Attention Enhanced CNN-BiLSTM hybrids, this framework intentionally removes the spatial CNN layers entirely and strictly builds on projection of Query (Q), Key (K) and Value (V) matrixes directly from the recurrent hidden state h_t to

dynamically weigh important packet-sequences without ballooning parameter size above edge restriction. The attention weights α_t are given by:

$$\alpha_t = \text{softmax}(Q \cdot K^T / \sqrt{d_k})$$

The context vector c_t is given by:

$$C_t = \sum_j \alpha_{t,j} V_j$$

To obtain output result, the Target hidden state is amalgamated with the Context vector. Here, to curb overfitting, the model is regularised by Dropout (rate=0.3) [16].

Projected Gradient Descent (PGD-10) Adversarial Training

The Projected Gradient Descent (PGD-10) Adversarial Training is a model of Module 5.

It defends against evasion attacks through strong minimax optimization loop, whose training should follow:

$$\min_{\theta} E_{(x,y)} \mathbb{D}[\max_{\delta \in S} L(\theta, x + \delta, y)]$$

Based on Madry et al [6]., the framework implements Madry's PGD-10 method to capture the case of a strong, white-box threat actor [6]. The inner maximisation problem generates the adversary perturbation δ with L^∞ norm within a radius

$$\epsilon = 0.05$$

$$S = \{\delta \mid \|\delta\|_\infty \leq \epsilon\}$$

The adversarial agent is repeatedly applying subtle first-order perturbations via gradients of classifiers cross-entropy loss about 15-dimensional networks flow vector by tweaking variables about most critical packet parameters like packet payload's size and its inter-arrival times across ten iterations. The perturbation updates as follows for current perturbed sample to next perturbed sample x^{t+1} :

$$x^{t+1} = \Pi_{x+S}(x^t + \alpha \cdot \text{sign}(\nabla_x L(\theta, x^t, y)))$$

With an iteration α step size $\epsilon/4 = 0.0125$ and Π used as projection operator ensuring that perturbed vectors are mapped back to $L^\infty [0, 1]$; constraint space after clipping. The new "malicious" packets are tweaked to bypass decision boundaries set up by the networks by doing so.

$$\Pi_{x+S}(x') = \text{Clip}_{[0,1]}(\max(x - \epsilon, \min(x + \epsilon, x')))$$

During the training process 50% samples for given batch consists of "perturbed samples" (or "failed to correct samples" as per PGD-10) generated on fly. Model parameters θ are then updated through gradient descent based on the mixed set of results

Explainable AI (XAI)

In response to the poor transparency characteristics of Deep sequence models, as also recently implemented for cyber-security anomaly detection, the proposed work integrates the Game-Theoretic KernelExplainer directly in the framework [4]. This module computes the marginal contribution of features leading to the alerts by

computing the mean marginal contribution feature explanations i.e. Shapley values upon a stratified background distribution. This translates the opaque tensor operations into verifiable attributions with the help of an additive explanation model:

$$g(z') = \varphi_0 + \sum_{i=1}^M \varphi_i z'_i$$

The Shapley values are: φ_i is the Shapley value of feature I and z' is the coalition vector. These Shapley values are based on an assumed background sample of 100 stratified samples to compute the Shapley values over all feature coalitions..

3.3 Project Plan

The development process occurred over 36 weeks and featured these four phases:

Phase 1: Pre-study : literature review, environment setup and IoTID20 analysis

Phase 2: Feature Engineering: PI, Pareto discovery and CREA algorithms implementation highlighted

Phase 3: Modeling : Custom Attention-BiLSTM architecture development, Clean baseline training

Phase 4: Adversarial Training : Implementation of FGSM/PGD-10 loops and PGD adversarial training (Goodfellow et al. [5], Madry et al. [6], Tsipras et al. [34])

3.4 Task Allocation

This table depicts the timeline of the principal activities in each period of the project, from week 12 to week 48.

Tasks	Weeks																		
	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Pre-Study																			
Feature Engineering																			
Modeling																			
Adversarial Training																			

3.5 Summary

This chapter presented the system design and implementation methodologies used for researching the proposed system. CREA+alignment, Luung Self-Attention and PGD-10 adversarial training as a Class-balanced feature selection are the pipelines designed and satisfy the edge optimization requirements. Each pipeline uses robustness and explainability which are applied consistently.

Chapter 4

Implementation and Results

Chapter 4 describes the implementation and results for this research.

Here are some hardware, software and security explanations: adversarial robustness, explanation and some detailed breakdowns of performances obtained when processing the data after implementation. This is described as a quantitative approach to how some software and results. This will be shown by means of hardware data.

4.1 Environment Setup

The implementation of this project has occurred as a team work project. Each of the different aspects such as for the development of framework training and for validation for the auditing explainability:

Component	Specification
Operating System & Base Interpreter	The Python interpreter is: Operating System & Base Interpreter: Microsoft Windows 11 Professional (64-bit), Python Interpreter Version: Python 3.10.12.
Deep Learning Framework	We have implemented it using Deep Learning Framework (DLF) TensorFlow 2.10.0 and Keras 2.10.0 on Windows hosts with native GPU execution. NVIDIA CUDA Toolkit 11.2 and cuDNN 8.1 were used for the GPU acceleration, which allowed the calculation of the gradients on the GPU during PGD-10 perturbation sweeps.
Scientific Computing & Data Wrangling	For serialization of tabular metrics and stratified partitioning as well as scaling with MinMaxScaler and encoding labels, we have used the scientific computing & data wrangling libraries NumPy 1.23.5, Pandas 1.5.3 and Scikit-learn 1.2.2.
Feature Selection & Oversampling	For Permutation Importance calculation, XGBoost is used at 1.7.3 level while for the SMOTE training split over-sampling, Imbalanced-learn is used at the 0.10.1 level.
Explainable AI & Visualization	Explainable AI & Visualization: KernalExplainer calculations: added the multi-class version of SHAP (0.42.1) [updated] and Matplotlib (3.7.1) & Seaborn (0.12.2) for plotting performance curves.
Hardware Setup	Setup CPU: Ryzen 9 7900X (12 Physical Cores, 24 Logical Cores, 4.7 GHz, Turbo Up to 5.3 GHz), RAM: 32 GB (Dual Channel DDR5 6000MHz) Memory, GPU: NVIDIA GeForce RTX 4060 (8 GB VRAM).

Since out-of-memory error can arise due to the large memory usage when training the neural network models, the use of DYNAMIC GPU memory growth was imposed (to allocate the VRAM as per demand). Also the virtual environment (python virtualenv) was created an isolate and separate from main Python distribution

there will be no chance of conflicting..

4.2 Testing and Evaluation/Performance/ Comparative Analysis

To [In the process of evaluating the proposed solutions (i.e., defense mechanisms), an evaluation was performed by mainly using below three aspects - namely, (1) Computational Efficiency (parameter count, model size on disk in Mbs and mean inference latency in ms per sample) [0.0, 0.01, 0.05, 0.1, 0.2, 0.3]; (2) Clean-Performance (Accuracy, Macro-Averaged F1-Score and per-class F1-scores); and (3) Adversarial Robustness (FGSM and PGD-10 attack by using perturbation magnitude of ϵ)

The pipeline generated 22 core output artifacts in outputs_phase2 in exact chronological order:

#	Artifact	Size	Description
1	module1_checkpoint.pkl	500.17 MB	Serialized raw training/testing splits after leakage dropping, normalization, and label encoding.
2	module1_log.json	2.35 KB	Logs metadata on raw data dimensions, target column structures, split validation ratios, and execution times.
3	class_distribution_multiclass.png	45.40 KB	Bar chart of the target class imbalance (e.g., majority class > 66% of the data, minority MITM Spoofing < 2%).
4	module2_checkpoint.pkl	528.75 MB	Serialized permutation importance matrices, surrogate model weights, and out-of-bag validation metrics.
5	module2_log.json	0.42 KB	Configuration file documenting XGBoost surrogate hyperparameters during feature selection.
6	pi_ranking_all_features.png	231.20 KB	Horizontal bar plot of permutation importance scores for all 69 features.
7	pi_ranking_balanced.csv	3.59 KB	Tabular file listing all 69 features sorted in descending order of class-weighted permutation importance.
8	xgb_perclass_sweep.pkl	7.36 KB	Binary dictionary of XGBoost per-class surrogate validation F1-scores.
9	xgb_perclass_sweep.csv	13.36 KB	Tabular format of XGBoost surrogate metrics across classes.
10	k_candidates.json	0.42 KB	Configuration defining feature subset candidate values.

11	k_discovery_perclass_curves.png	277.56 KB	Line plot of model metrics against feature subset sizes.
12	pareto_curve_data.csv	2.94 KB	Tradeoffs between model footprint and F1-macro.
13	pareto_curve_with_kstar.png	137.89 KB	Pareto frontier plot showing the optimal k=15 feature selection.
14	k_optimal_selection.json	2.23 KB	Configuration of features selected at k=15.
15	module3_checkpoint.pkl	500.21 MB	MinMaxScaler and CREA projection matrices.
16	datahub_checkpoint.pkl	4.65 GB	Master checkpoint containing the complete multi-class 3D tensors.
17	feature_ablation_comparison.csv	0.87 KB	Tabular performance metrics for $k \in [4, 15, 34, 69]$.
18	dimensionality_ablation.csv	0.50 KB	Comparison metrics for Std PCA, PI-only, and proposed PI+CREA projection.
19	architectural_ablation.csv	0.52 KB	Validation metrics comparing unidirectional LSTM, standard Bi-LSTM, and Attention-BiLSTM.
20	master_performance_matrix.csv	1.74 KB	Table comparing standard evaluation metrics across all 12 model configurations.
21	fgsm_results.json	11.87 KB	Performance log under Fast Gradient Sign Method (FGSM) attacks.
22	pgd_robustness_results.csv	0.96 KB	Robustness metrics recorded during Projected Gradient Descent (PGD-10) sweeps.

4.3 Results and Discussion

Results are presented as a Three-Tier Success Narrative: (1) baseline clean performance is used to validate the feature reduction pipeline, (2) show the attackability of undefended model, (3) tradeoff of PGD adversarial train is used as the finish engineer output.

Feature Selection & Dimensionality Reduction: The PI is based on Class-Balanced PI has reduced the original 69 features to 15, reducing 78.3% of the features [2]. The Pareto (via Kneedle inflection algorithm) sweep was performed to get the k at inflection, k=15 [32]

Table 4.1: Selected 15-Feature Subset

Rank	Feature Name	Description	Category
1	Flow_Duration	Total duration of the network flow in microseconds	Temporal
2	Src_Port	Source port number identifying the origin application	Network
3	Dst_Port	Destination port number identifying the target application	Network
4	Init_Bwd_Win_Byts	Number of bytes sent in the initial backward TCP window	Size
5	Flow_Pkts/s	Number of packets transmitted per second	Rate
6	Protocol	Transport layer protocol (e.g., TCP, UDP)	Network
7	Flow_IAT_Max	Maximum inter-arrival time between two packets	Temporal
8	Flow_IAT_Mean	Mean inter-arrival time between consecutive packets	Temporal
9	Flow_IAT_Min	Minimum inter-arrival time between two packets	Temporal
10	Bwd_Header_Len	Total header byte length in the backward direction	Size
11	Pkt_Len_Mean	Average byte length of packets in the flow	Size
12	Fwd_Header_Len	Total header byte length in the forward direction	Size
13	Pkt_Size_Avg	Average size of the packets across the entire flow	Size
14	Idle_Mean	Average time the flow was idle before becoming active	Temporal
15	ACK_Flag_Cnt	Number of packets with the Acknowledgment flag set	Flag

Tier 1 - Baseline Test- The Victory in Clean Data: The 78.3% feature reduction (69 to 15 features) could preserve and improve classification outcome. The victory proved the edge readiness of compressed pipeline.

Detection task 1 -Binary classification between (Normal and Attack): Standard attack detector(Bi-LSTM with all 69 features-B1) versus compressed detector (Bi-LSTM on PI+CREA subset (15 features)-B2):

Table 4.2: Binary Classification — Regular Bi-LSTM (69 Features) vs. Optimized Bi-LSTM (15 Features, PI+CREA)

Metric	Regular Bi-LSTM (B1)	Optimized Bi-LSTM (B2)	Change
Features	69	15	-78.3%
Accuracy	99.64%	99.81%	+0.17%
Precision	0.9982	0.9988	+0.0006
Recall	0.9979	0.9992	+0.0013

F1-Score	0.9981	0.9990	+0.0009
Parameters	367,233	311,937	-15.1%
Model Size	4.26 MB	3.63 MB	-14.8%
Training Time	163.5 s	270.1 s	+65.2%
Inference Latency	0.0263 ms	0.0247 ms	-6.1%

After optimisation the Bi-LSTM (B2) outperformed B1 across every aspect despite being trained with only 15 out of originally 69 features. 54 redundant features were removed from the input data resulting in less noise and clearer separation boundaries making the model better able to infer outputs. Model footprint dropped 14.8% (4.26 → 3.63 MB) and latency 6.1% (0.0263 → 0.0247 ms per sample), confirming the pipeline is ready for edge deployment.

Multi-Class Classification (5-Class): To analyse the impact of feature selection and optimisation on the five class problem (Normal, Mirai, DoS, Scan, MITM) a regular Bi-LSTM on all 69 available features (F1) is compared with the optimised Bi-LSTM on the 15 features selected by PI+CREA (F4):

Table 4.3: Multi-Class Classification — Regular Bi-LSTM (69 Features) vs. Optimized Bi-LSTM (15 Features, PI+CREA)

Metric	Regular Bi-LSTM (F1)	Optimized Bi-LSTM (F4)	Change
Features	69	15	-78.3%
Accuracy	89.44%	92.80%	+3.36%
Precision	0.8194	0.8765	+0.0571
Recall	0.9079	0.9654	+0.0575
F1-Macro	0.8506	0.9032	+0.0526
F1-Worst (MITM)	0.5267	0.6521	+0.1254
Parameters	367,749	312,453	-15.0%
Model Size	4.26 MB	3.63 MB	-14.8%
Training Time	142.6 s	191.9 s	+34.6%
Inference Latency	0.0305 ms	0.0265 ms	-13.1%

The optimized Bi-LSTM (F4) shows a performance improvement from F1 in all metrics. A +3.36% (accuracy) and +0.0526 (F1-macro) confirm that the class-balanced feature selection CREA eigenspace alignment eliminates noisy features, improving multi-class separability. The minority MITM F1-worst score increased by +0.1254, which confirms that class-weighted permutation importance ensures preservation of features critical to rare attack detection.

With the transition attention-biLSTM architecture (A3_NODEF, undefended), clean multi-class accuracy is now **92.62%**, 0.9011 (F1-macro) at (1.49 MB device size and 0.0357 ms latency) - as luong self attention effectively learns the longer-range dependencies between packets while still ensuring it does not exceed the < 2.0MB device size constraint.

Architectural Ablation: LSTM, Bi-LSTM and Attention-BiLSTM models were compared at the same PI+CREA (15 features) input [10, 11, 14]

Table 4.4: Architectural Ablation — Sequence Model Comparison

Model ID	Architecture	Accuracy	F1-Macro	F1-Worst (MITM)	Parameters	Model Size	Latency (ms)
A1	Standard LSTM (Unidirectional)	92.07%	0.8941	0.6355	123,461	1.45 MB	0.0222
A2 (=F4)	Standard Bi-LSTM	92.80%	0.9032	0.6521	312,453	3.63 MB	0.0265
A3	Attention-BiLSTM (Proposed)	92.62%	0.9011	0.6456	378,245	1.49 MB	0.0357

Attention-BiLSTM (A3) matches the standard Bi-LSTM (A2) while using a 1.49 MB file versus 3.63 MB — the only architecture satisfying the < 2.0 MB edge constraint.

Feature Ablation: For discovering optimal feature subset our automated pipeline mathematically found best candidates at $k \in [4, 15, 18, 34, 69]$. During initial XGBoost surrogate sweep, K=18 was temporarily identified as a candidate because of its peak in harmonic score were evaluated using the Kneedle inflection threshold [32]:

Table 4.5: Feature Ablation — Impact of Feature Subset Size on Multi-Class Performance

Model ID	Features (k)	Feature Method	Accuracy	F1-Macro	F1-Worst (MITM)	Model Size
F1	69	None (Full Features)	89.44%	0.8506	0.5267	4.26 MB
F3	4	PI Knee Point	89.85%	0.8738	0.6027	3.50 MB
F2	15	PI Top-15 (No CREA)	90.67%	0.8816	0.6446	3.63 MB
F6	15	PI Only (Control)	90.28%	0.8762	0.6414	3.63 MB
F4	15	PI+CREA (Proposed)	92.80%	0.9032	0.6521	3.63 MB
F5	34	PI+CREA Pareto	93.07%	0.9055	0.6623	3.85 MB

PI+CREA at k=15 gives a +2.52% accuracy improvement over PI-only (F6: 90.28% → F4: 92.80%), confirming the CREA eigenspace transform aligns feature distributions to improve separability and corroborating the 15-feature optimum reported by Sadhwani et al. [36]. k=34 (F5) adds only 0.27% accuracy while doubling the footprint, making it unsuitable for edge deployment.

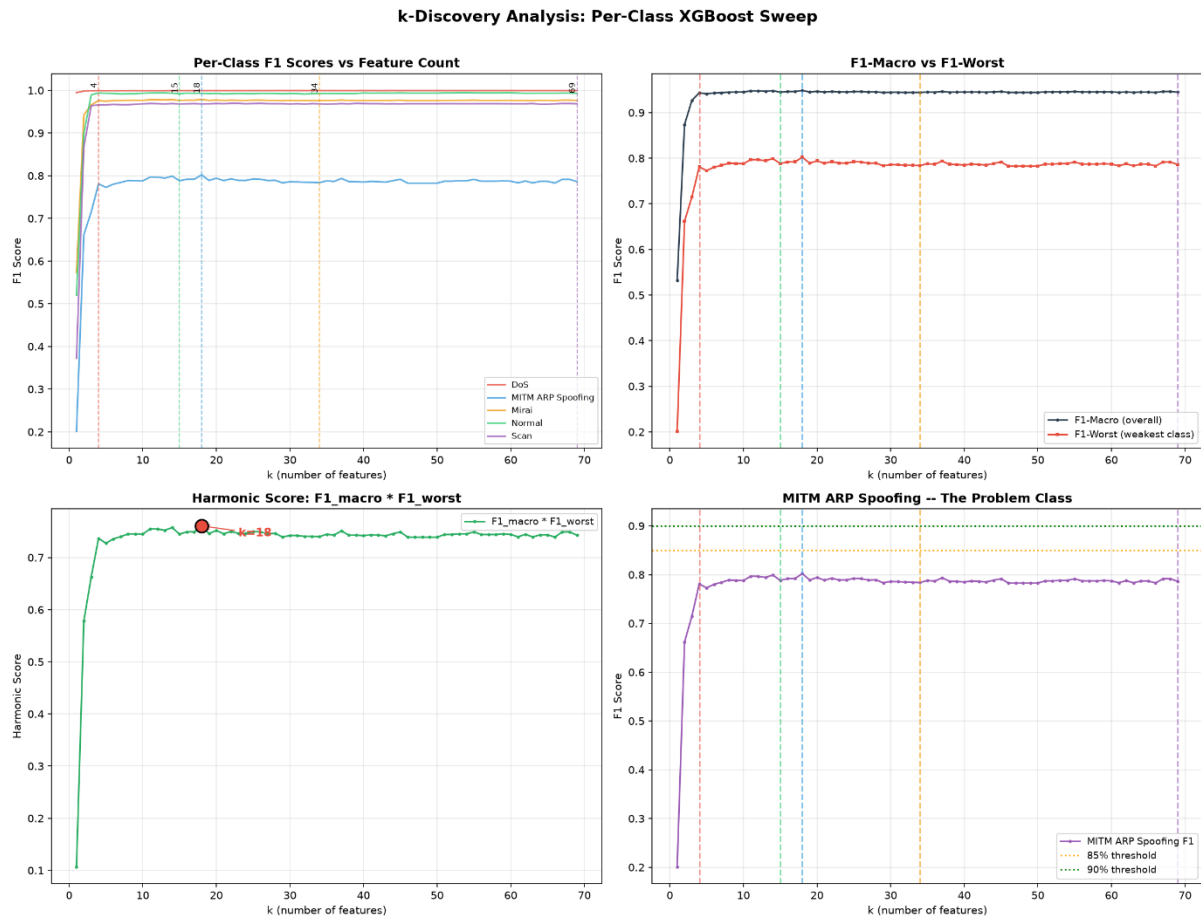


Figure 4.3: Per-class validation $F1$ -macro curves across feature subset sizes k ranging from 1 to 69.

Candidate Discovery Mathematical (XGboosts Surrogate Sweep)

To discover some candidates automatically as opposed to arbitrarily hard-coding them in, a fast XGboost surrogate model was swept across the values of all possible $k \in [1,69]$, taking the Harmonic Score average score of both $F1$ -Macro and $F1$ Worst - so that Minority classes such as MITM ARP Spoofing were not disregarded, some of these candidates included the 5 candidates chosen automatically by the Discovery algorithm.

Table 4.5.1: XGBoost Surrogate Candidate Discovery

k Features	Mathematical Criterion	F1-Macro	F1-Worst (MITM)	Harmonic Score
4	k_knee (Pareto Inflection)	0.9434	0.7811	0.7369
15	k_phase1 (Baseline target)	0.9450	0.7882	0.7449
18	k_best_harmonic (Peak Surrogate)	0.9484	0.8021	0.7607
34	k_pareto_f1 (Peak Surrogate F1)	0.9442	0.7838	0.7401
69	k_full (All Features)	0.9448	0.7862	0.7428

Full Bi-LSTM Sequence Validation and Final Selection : While $k = 18$ scored highest within the context of the harmonic score for the shallow surrogate sweep, the real test was to prove the chosen five Candidates on the full Bi-LSTM sequence classifier. The results of the final structure based validation are shown below:

Table 4.5.2 Full Bi-LSTM Validation and Final Selection

k Features	Accuracy	F1-Macro	F1-Worst (MITM)	Harmonic Score	Train Time (s)	Latency (ms)
15 (Selected)	94.00%	0.9152	0.6996	0.6402	225.0	0.0272
34	93.40%	0.9098	0.6899	0.6277	265.8	0.0264
69	93.53%	0.9098	0.6891	0.6269	274.3	0.0269
4	92.13%	0.8951	0.6433	0.5759	168.1	0.0261
18 (Rejected)	92.17%	0.8958	0.6329	0.5670	157.2	0.0263

Selection Rationale: The full sequence validation categorically proved that the Bi-LSTM model drew better separating multi-class boundaries with 15 Features. The candidate of $k = 18$ that did great on tree-based shallow surrogate, plummeted to the lowest harmonic score of 0.5670 using deep sequential processing. The selection criteria of $\text{argmax}(\text{F1_macro} * \text{F1_worst})$ thus locked in $k = 15$ as the absolute optimal number of feature counts for the final edge deployment framework.

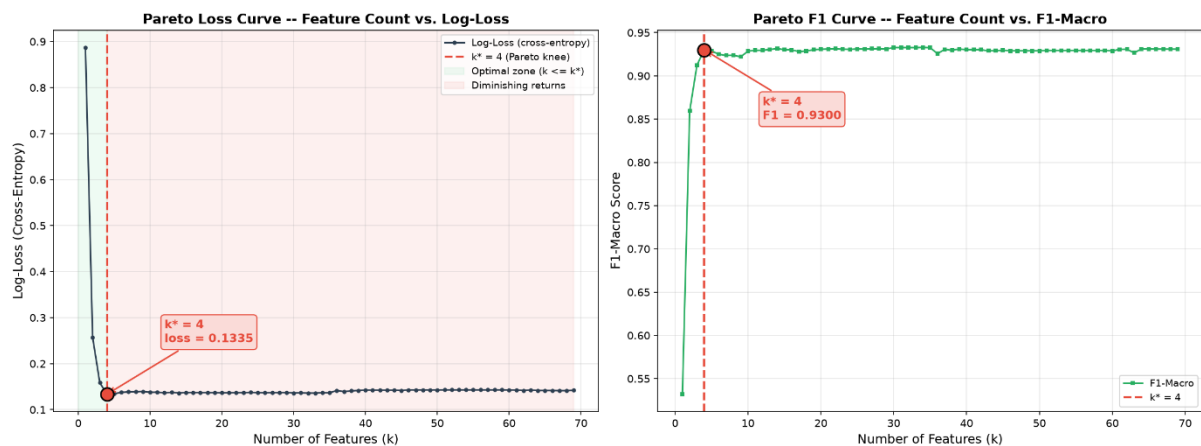


Figure 4.4: Pareto frontier curve illustrating between macro-F1 performance and model storage footprint.

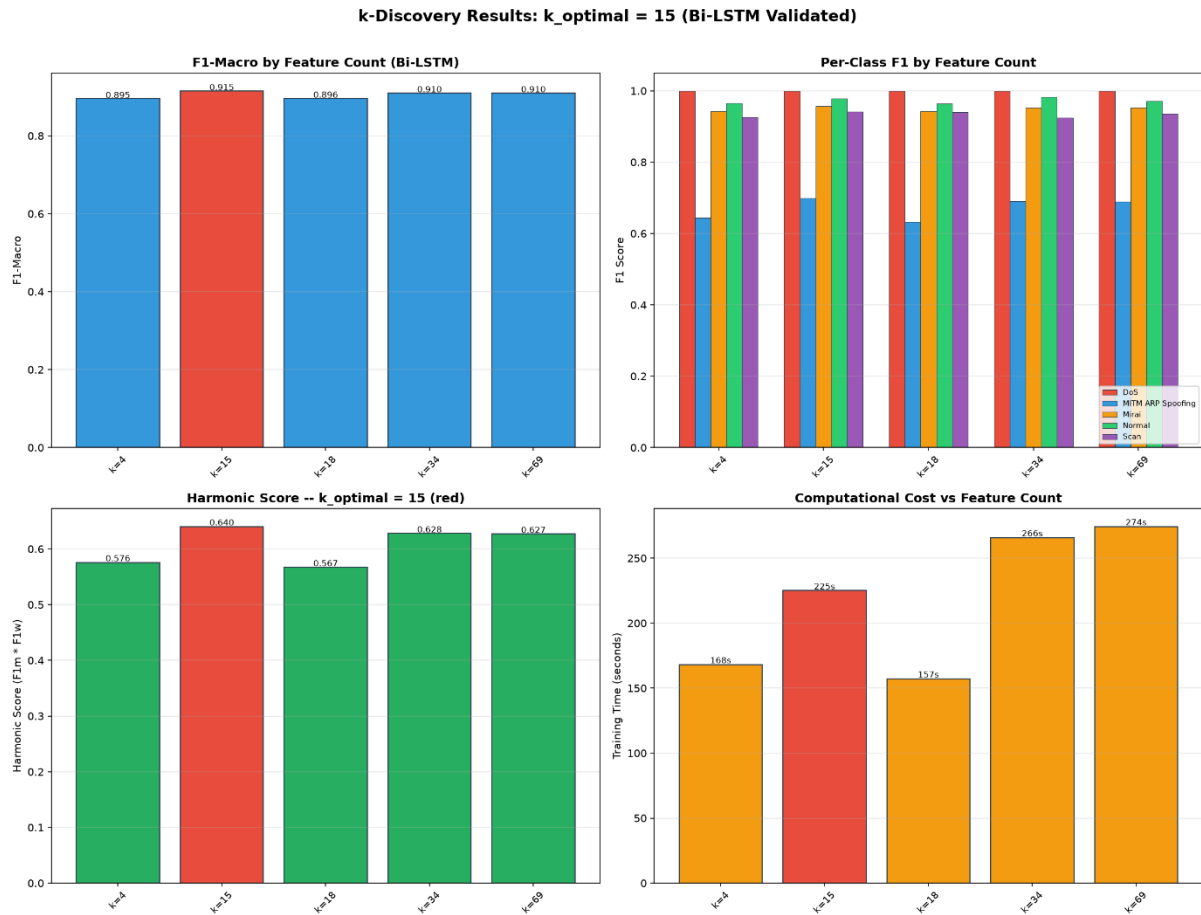


Figure 4.5: Accuracy and macro-F1 comparison for feature subsets k in $[4, 15, 18, 34, 69]$ across evaluated architectures.

Dimensionality Reduction Method Comparison:

Table 4.6: Dimensionality Reduction Comparison (All at $k=15$ Features)

Model ID	Reduction Method	Accuracy	F1-Macro	F1-Worst (MITM)	Semantic Interpretability
D1	Standard PCA (69 $\rightarrow$ 15)	94.92%	0.9258	0.7393	No (abstract components)
D2	PI Only (No Transform)	90.35%	0.8776	0.6232	Yes
F4	PI + CREA (Proposed)	92.80%	0.9032	0.6521	Yes

Standard PCA (D1) creates components by linear combination of all 69 features - this results in maximum possible raw accuracy of 94.92%, sacrificing human interpretable meaning of these components [24]. This would block any attempt to apply posthoc SHAP explainability techniques to the model as no human analyst would understand how a combination of 68 factors relates to the model's prediction [4]. The PI+CREA method preserves human interpretable meaning at comparable accuracy (8) [3].

Performance Matrix (Full Result Table for all model configurations):

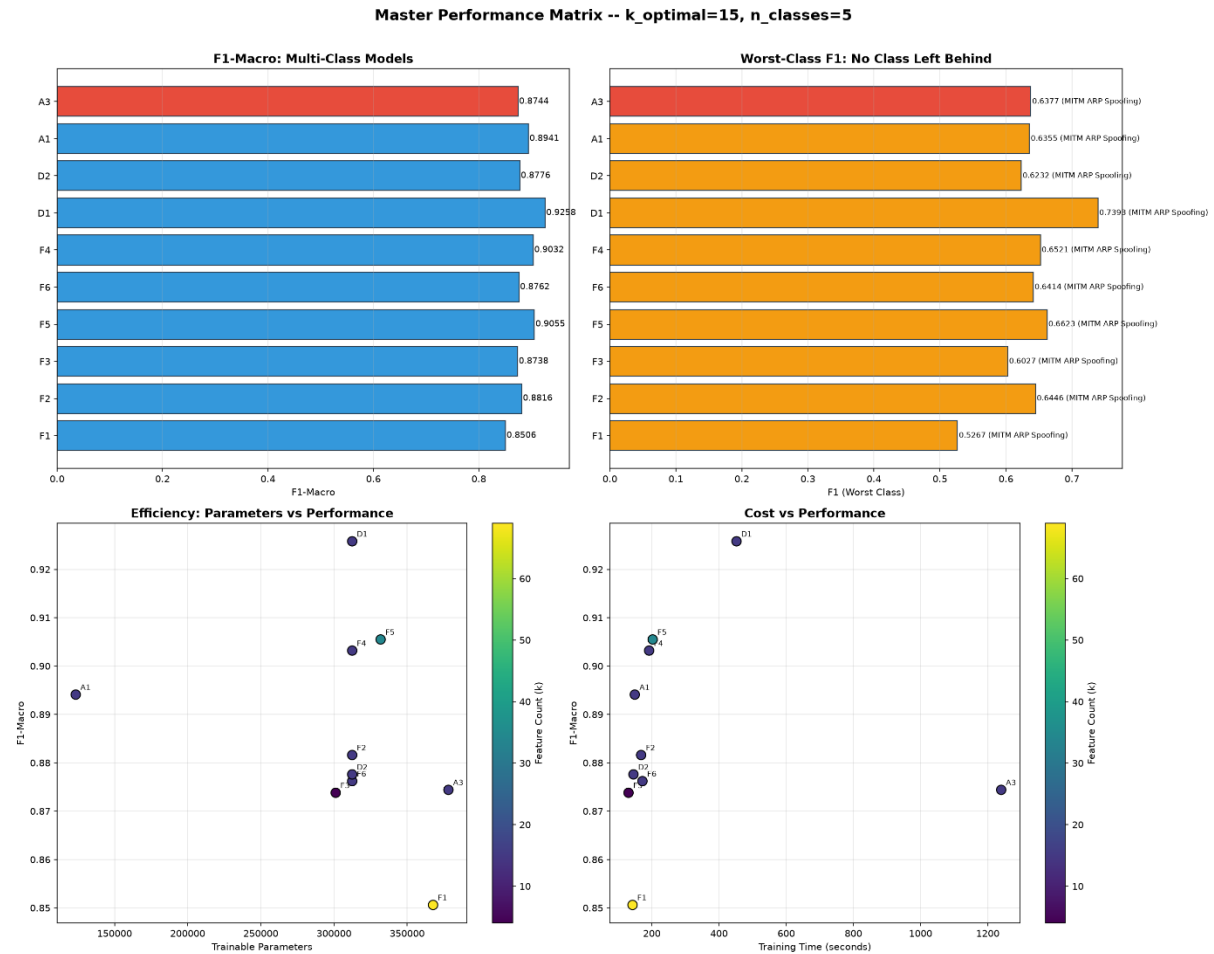


Figure 4.6: Master performance visualization of standard classification metrics across all model configurations.

Table 4.7: Master Performance Matrix — All 12 Model Configurations

Model	Task	k	Arch.	CREA	Acc.	F1-Macro	F1-Worst	Params	Size	Lat. (ms)
B1	Binary	69	Bi-LSTM	No	99.64%	0.9981	—	367,233	4.26 MB	0.0263
B2	Binary	15	Bi-LSTM	Yes	99.81%	0.9990	—	311,937	3.63 MB	0.0247
F1	Multi	69	Bi-LSTM	No	89.44%	0.8506	0.5267	367,749	4.26 MB	0.0305
F2	Multi	15	Bi-LSTM	No	90.67%	0.8816	0.6446	312,453	3.63 MB	0.0270
F3	Multi	4	Bi-LSTM	No	89.85%	0.8738	0.6027	301,189	3.50 MB	0.0273
F4	Multi	15	Bi-LSTM	Yes	92.80%	0.9032	0.6521	312,453	3.63 MB	0.0265
F5	Multi	34	Bi-LSTM	Yes	93.07%	0.9055	0.6623	331,909	3.85 MB	0.0300
F6	Multi	15	Bi-LSTM	No	90.28%	0.8762	0.6414	312,453	3.63 MB	0.0266
D1	Multi	15	Bi-LSTM	PCA	94.92%	0.9258	0.7393	312,453	3.63 MB	0.0266
D2	Multi	15	Bi-LSTM	No	90.35%	0.8776	0.6232	312,453	3.63 MB	0.0277
A1	Multi	15	LSTM	Yes	92.07%	0.8941	0.6355	123,461	1.45 MB	0.0222
A3	Multi	15	Attn-BiLSTM	Yes	92.62%	0.9011	0.6456	378,245	1.49 MB	0.0357

Tier 2 - Adversarial Vulnerability (Catastrophic Failure of the Undefended Model):

Although the undefended Attention-BiLSTM (A3_NODEF) achieved a

commendable 92.62% clean multi-class accuracy, it harbored a critical structural vulnerability [9]. Standard classifiers often learn to associate labels with superficial statistical features like subtle changes in packet timing or size [5, 6], which are susceptible to small gradient-based perturbations.

A white-box PGD-10 attack at $\epsilon=0.05$ demonstrated this flaw, With undefended accuracy dramatically dropping from 92.62% to just **8.21%**. This reflected an 84.41% accuracy decrease, confirming that clean accuracy alone is an insufficient measure for security in IoT environments.

Tier 3 - The Strategic Robustness Tradeoff (PGD Adversarial Training): The Attention-BiLSTM was fortified through Adversarial Training with a PGD-10 loop [6]. A minimal concession in clean-accuracy (92.62%→**87.99%**; a 4.63% reduction) resulted in absolute structural resilience against attacks.

This accuracy-robustness tradeoff, a hallmark of robust optimization, plays a vital role [34]. Standard classifiers enhance accuracy by mimicking superficial correlations; adversarial training compels the network to discover robust structural features immune to gradient perturbations. In security-prone contexts, embracing this tradeoff is not just an option but a strategic imperative. A 87.99% clean accuracy is perfectly adequate for functional IoT device monitoring. However, a model rendered entirely vulnerable-achieving merely 8.21% accuracy under PGD-becomes obsolete.

Evaluation Under FGSM Attacks: The robustness of models was also assessed under white-box FGSM single-step attacks using various ϵ values: [5]:

Table 4.8: Model Performance Under FGSM Adversarial Attack

ϵ Magnitude	Undefended (A3_NODEF)	FGSM-Trained (A3_FGSM)	PGD-Trained (A3_PGD)	Binary Optimized (B2)
0.00 (Clean)	92.62%	90.32%	87.99%	99.81%
0.01	32.40%	77.72%	78.27%	48.39%
0.05	24.21%	95.96%	78.88%	44.09%
0.10	23.92%	71.61%	35.42%	44.49%
0.20	24.33%	40.96%	19.12%	47.51%
0.30	24.34%	34.88%	15.79%	46.19%

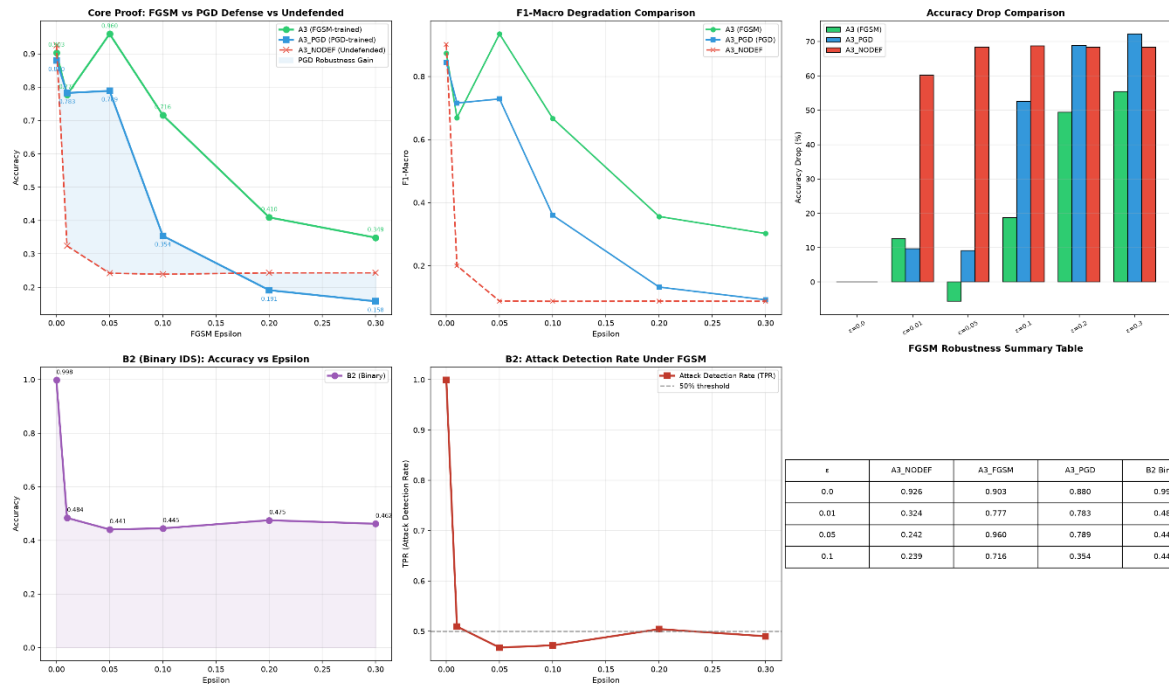


Figure 4.1: FGSM robustness curves showing accuracy degradation across perturbation magnitudes for all four evaluated models.

Catastrophic Overfitting (Observation 1): FGSM-trained model (A3-FGSM) spikes to 95.96% under FGSM which is above its clean accuracy of 90.32% under Epsilon=0.05. This is "catastrophic overfitting" to the single-step perturbation pattern where a model resists in the FGSM linear direction than the more difficult task of learning more robust features [5]. However under PGD accuracy crumbles to 44.04% where it faces a new iterative attack.

PGD-10 (Observation 1.1) Models subjected to strongest first adversarial order - that of [6] :

Table 4.9: Model Performance Under Projected Gradient Descent (PGD-10) Attack

ϵ Magnitude	Undefended (A3_NODEF)	FGSM-Trained (A3_FGSM)	PGD-Trained (A3_PG)	Binary Optimized (B2)
0.00 (Clean)	92.62%	90.32%	87.99%	99.81%
0.01	26.23%	77.00%	71.36%	37.89%
0.05	8.21%	44.04%	75.36%	26.66%
0.10	7.53%	17.85%	19.65%	24.50%
0.20	5.97%	3.89%	3.56%	26.74%
0.30	5.42%	2.10%	1.83%	27.35%

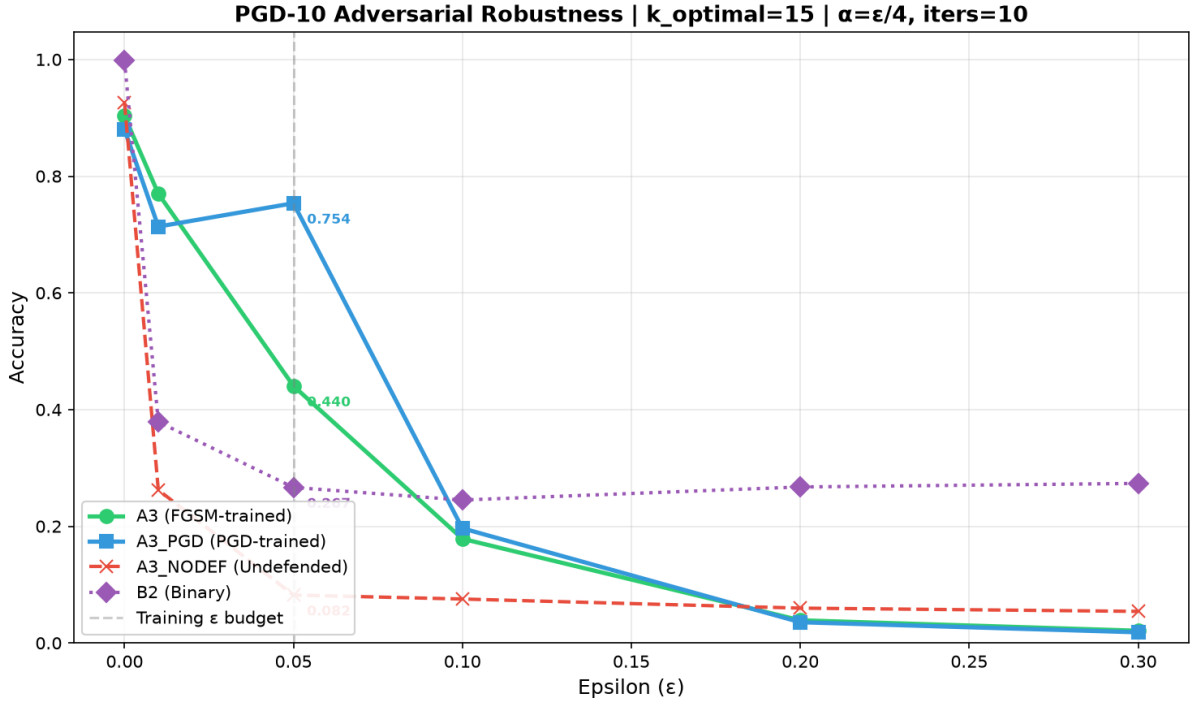


Figure 4.2: PGD-10 robustness curves comparing the undefended, FGSM-trained, and PGD-trained Attention-BiLSTM models across perturbation magnitudes.

PGD Robustness: With PGD-10 for the same $\epsilon=0.05$, the undefended model collapsed further to **8.21%** and the FGSM-trained model dropped lower to **44.04%** while A3_PG defended well at **75.36%** which is more than ten-fold performance for undefended model (**+67.15pp**) and (**+31.32pp**) for FGSM-trained. This indicates PGD adversarial training builds structurally robust boundaries and thus generalizes across multi-step perturbations while the FGSM-trained fails completely to PGD adversarial examples because the network has catastrophically overfitted it training-time single-step perturbations.

Table 4.10: PGD-10 Robustness Summary at $\epsilon=0.05$

Model Variant	Clean Accuracy ($\epsilon=0.00$)	PGD-10 Accuracy ($\epsilon=0.05$)	Accuracy Drop	Robustness Gain vs. Undefended
Undefended Attn-BiLSTM (A3_NODEF)	92.62%	8.21%	-84.41%	—
FGSM-Trained Attn-BiLSTM (A3_FGSM)	90.32%	44.04%	-46.28%	+35.83 pp
PGD-Trained Attn-BiLSTM (A3_PG)	87.99%	75.36%	-12.63%	+67.15 pp

Global and Local Explainability Analysis (SHAP): The SHAP KernelExplainer, gives feature-level interpretations at global and local scope, grounding the general tool specifically within botnet and NIDS environments [4] [38] [39].

Global Feature Importance: The SHAP global bar chart aggregates the mean absolute Shapley values across all test samples, identifying the features that most consistently drive threat classification globally [4] [21].

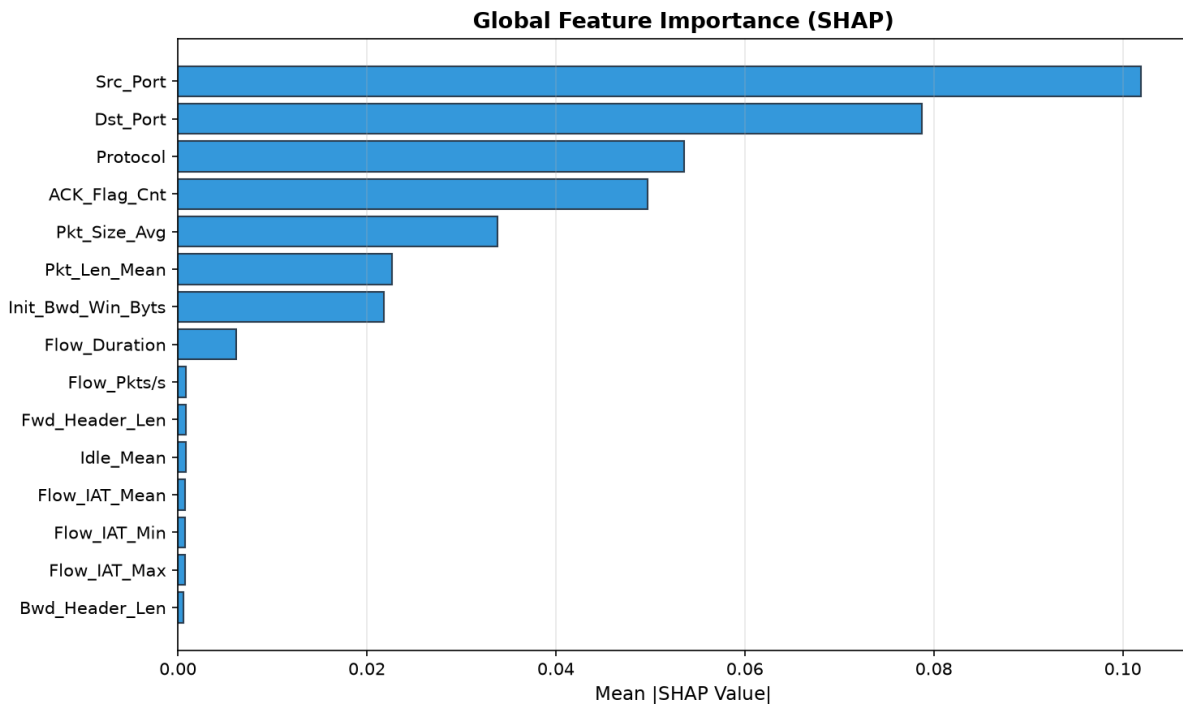


Figure 4.7: SHAP global feature importance bar chart showing the mean absolute Shapley values for the 15 selected features across all multi-class predictions.

Globally the highest-ranking features are Src_Port, Dst_Port, and Protocol, respectively. This demonstrates that structural network routing parameters and protocol types—rather than just the raw volume or duration of the traffic—are the strongest signatures for separating malicious anomalies from normal data.

Class-Specific SHAP Analysis: These visualizations show how each single variant of attack relies on different type of network -level signatures [4, 21]. Each dot represents a test sample instance. The colour indicated the value for specific feature (red indicates high, blue low), while the position indicates the shapley value (amount and direction of the contribution to model output for the given feature). Globally ranking highest for all variants of attacks have been src_port and dst_port and protocol. This mathematical ranking showed our class_balanced weighting had correctly avoided the typical volume-oriented features which are usually ranked through attacks- (total number of sent feature, flow duration -easily replicable by a savvy hacker nowadays). Instead, the method learned the importance of network addresses- and found manipulation of these features as the primary 'structural' signs indicative of when data could be singled out as malicious anomalies, that it had correctly separated from typical ones.

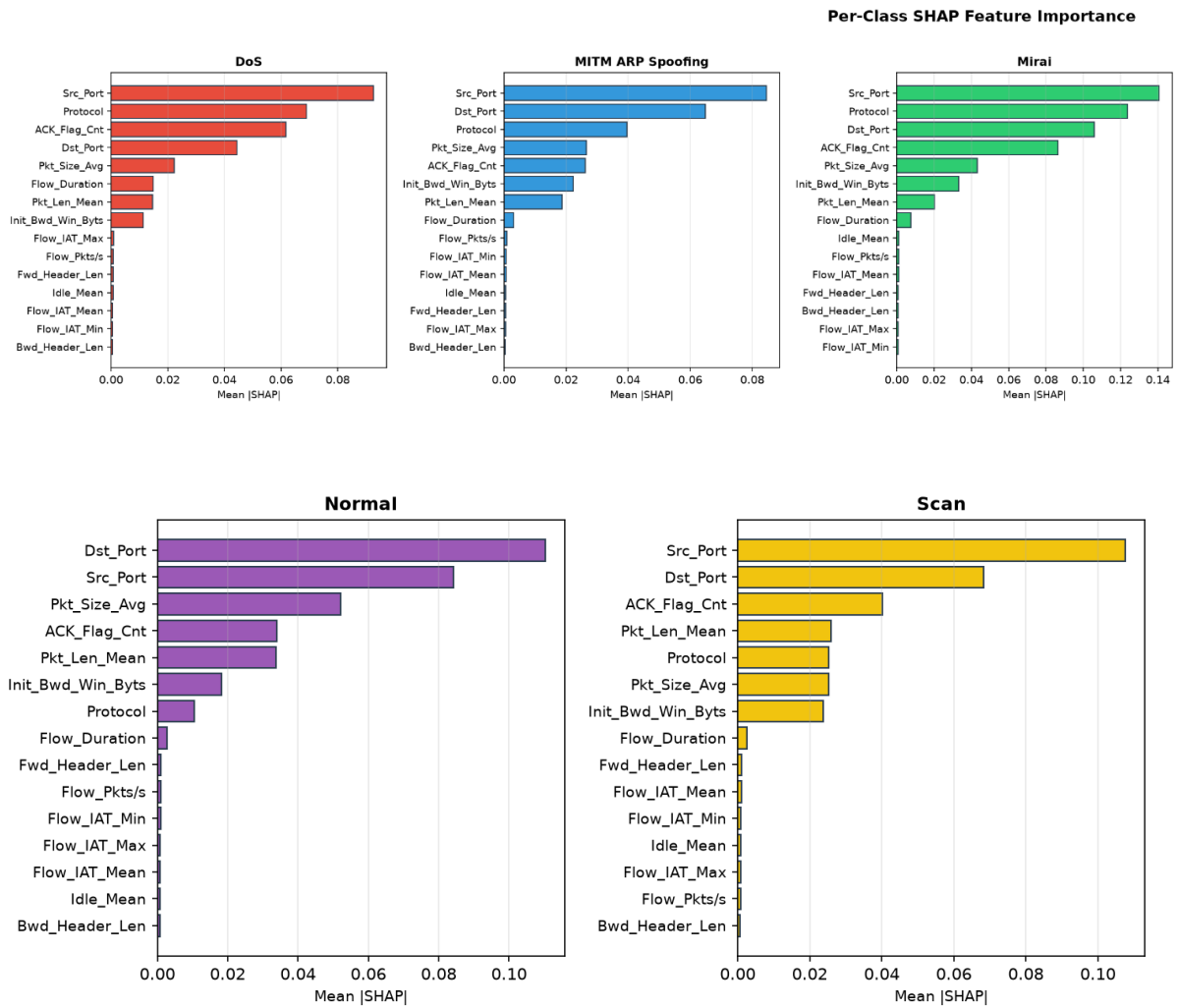


Figure 4.8: Per-class SHAP feature contribution bar chart summarizing importance across all attack classes.

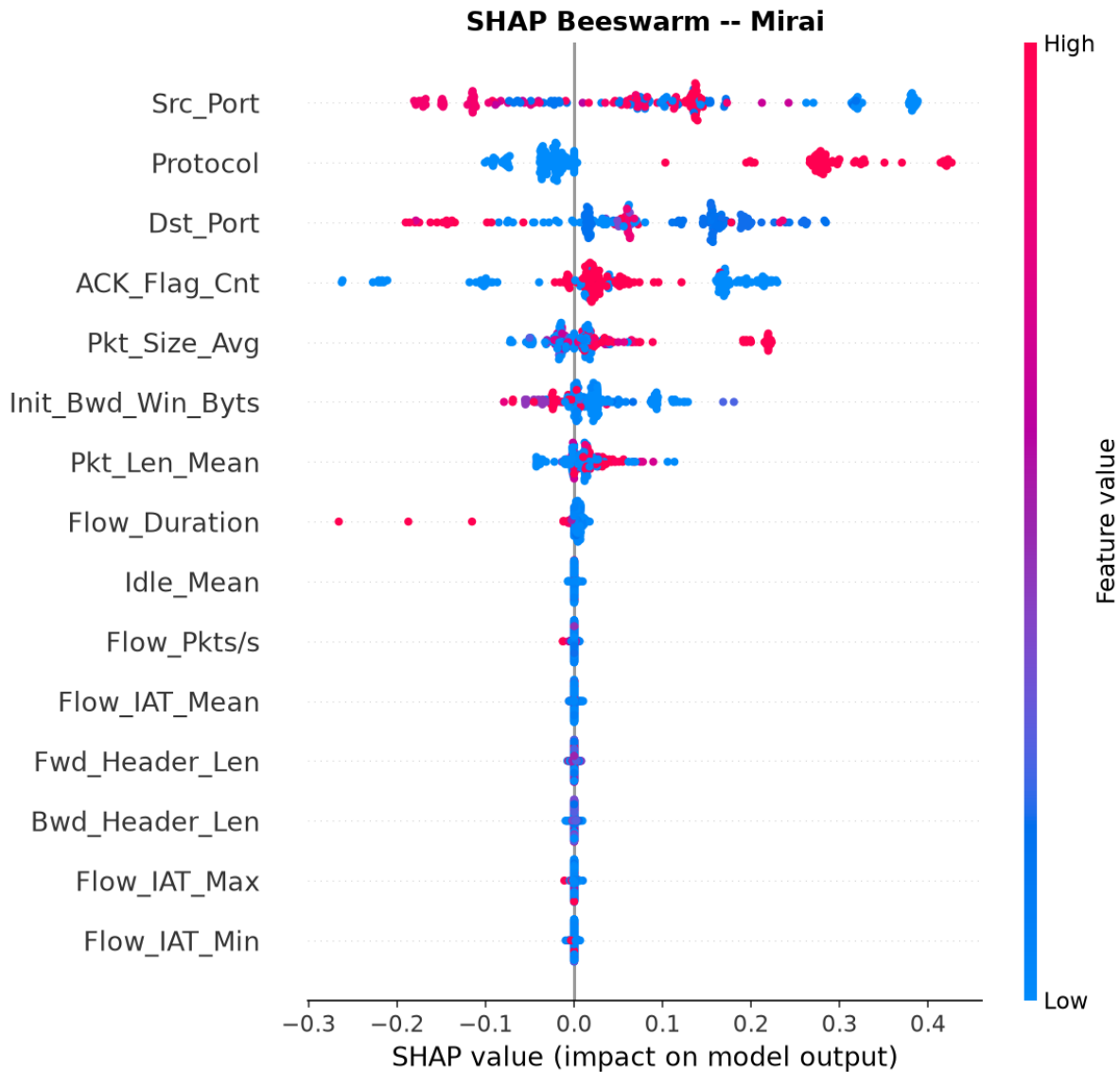


Figure 4.9: SHAP beeswarm plot for the Mirai class, illustrating how individual feature values drive the model's prediction toward or away from the Mirai classification.

For Mirai detection, the model relies heavily on specific structural network signatures, primarily driven by Src_Port (Impact: 0.1406), Protocol (Impact: 0.1236), and Dst_Port (Impact: 0.1059). This perfectly aligns with the real-world behavior of a Mirai botnet, which relies on algorithmic scanning of specific vulnerable destination ports over targeted protocols to establish Command and Control (C2) communication.

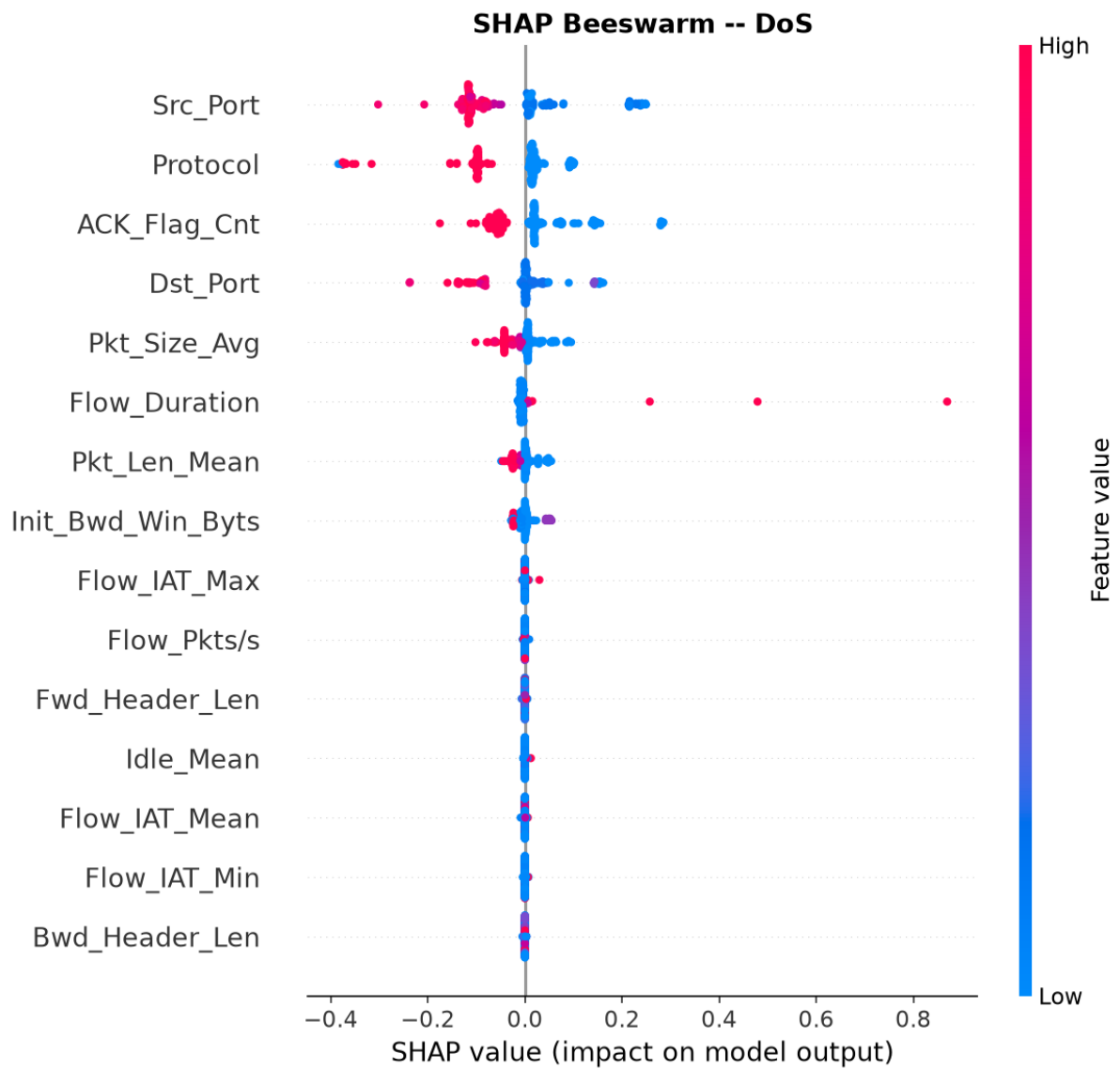


Figure 4.10: SHAP beeswarm plot showing feature-level attributions for the Denial of Service (DoS) attack class.

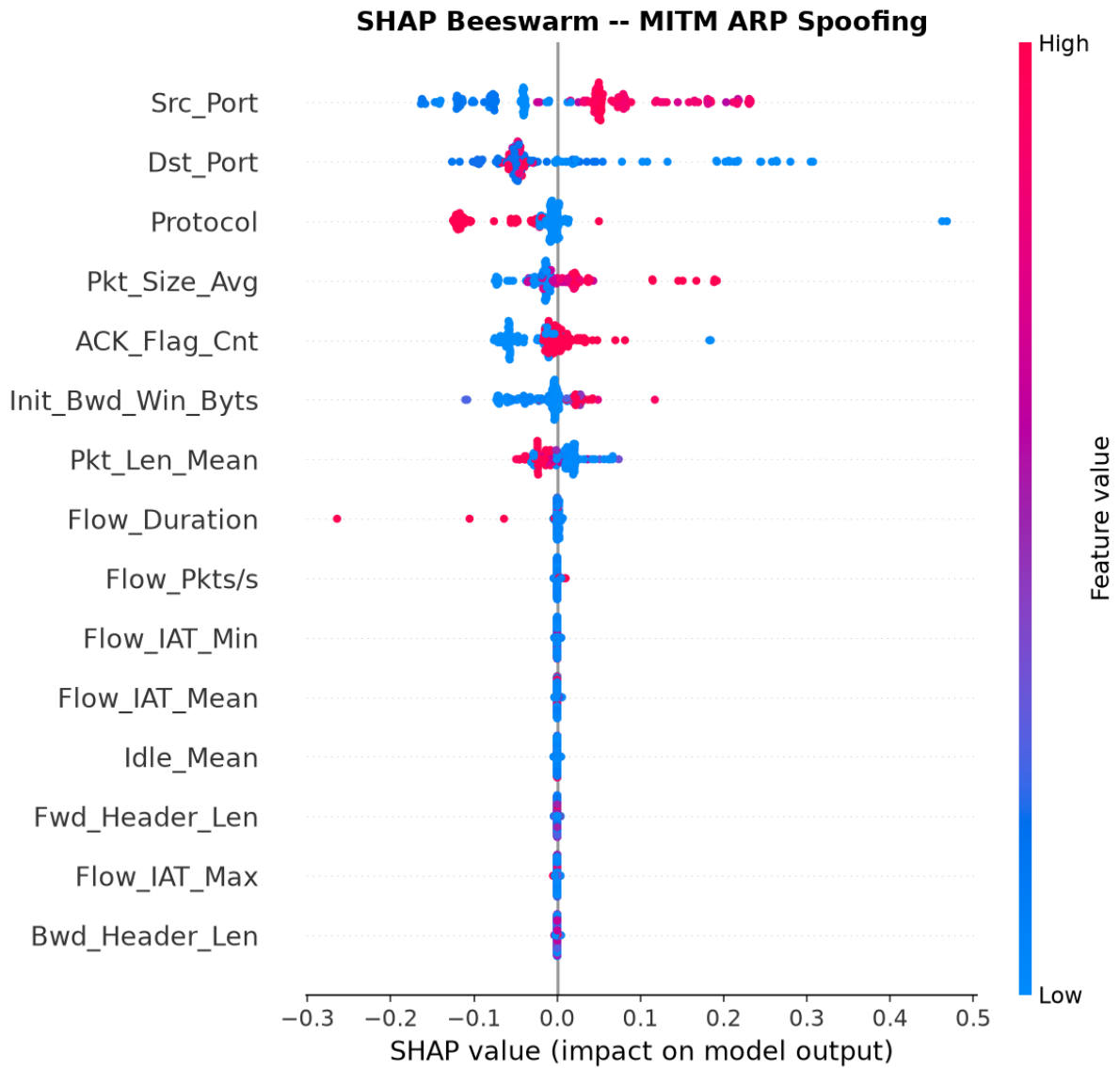


Figure 4.11: SHAP beeswarm plot showing feature-level attributions for the MITM ARP Spoofing attack class.

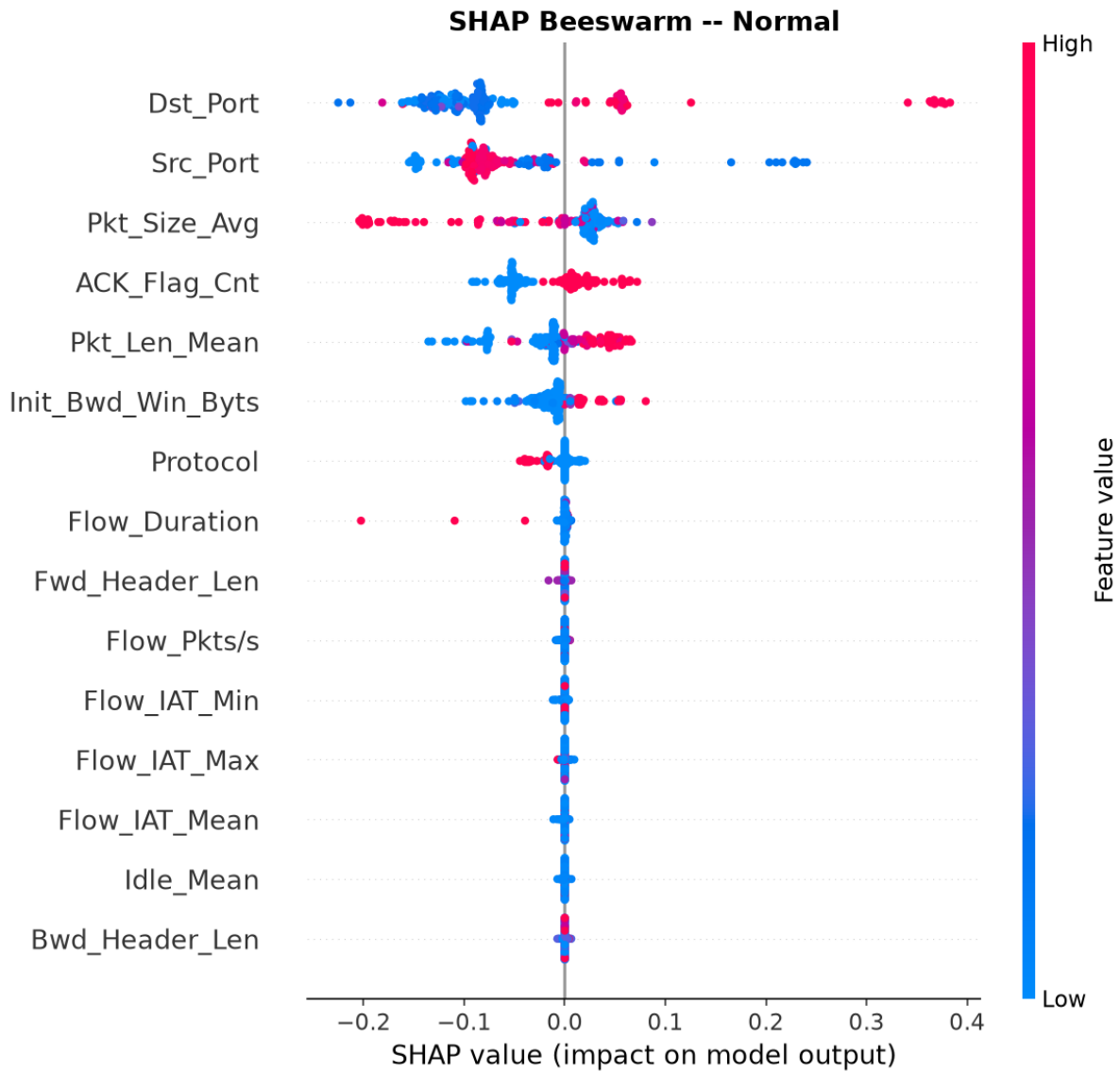


Figure 4.12: SHAP beeswarm plot showing feature-level attributions for the Normal traffic class.

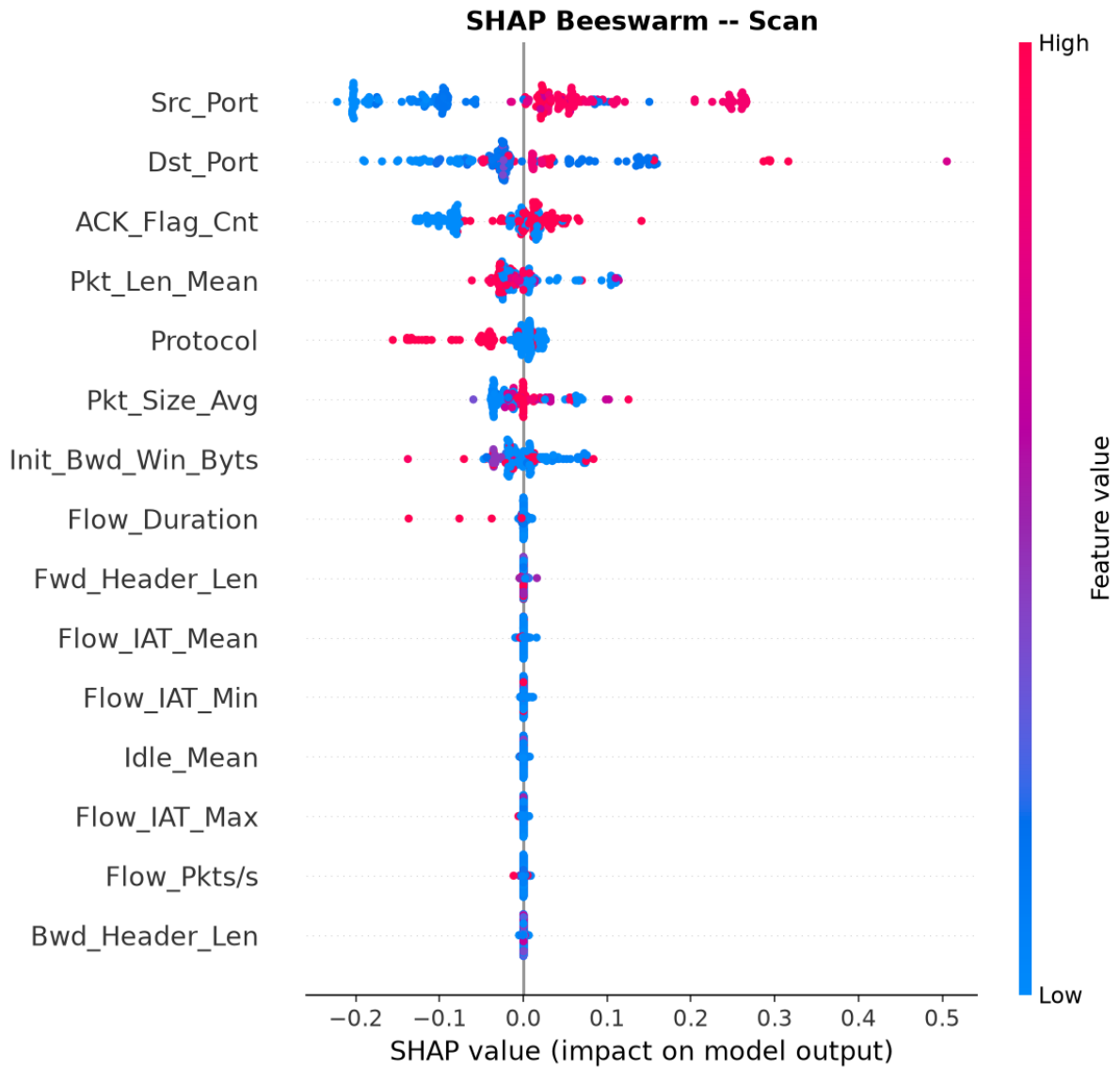


Figure 4.13: SHAP beeswarm plot showing feature-level attributions for the Port/Host Scan attack class.

Local Packet-Level Explanation: For each threat alert, a local SHAP waterfall chart deconstructs the prediction into the exact feature contributions that drove the decision [4, 21].

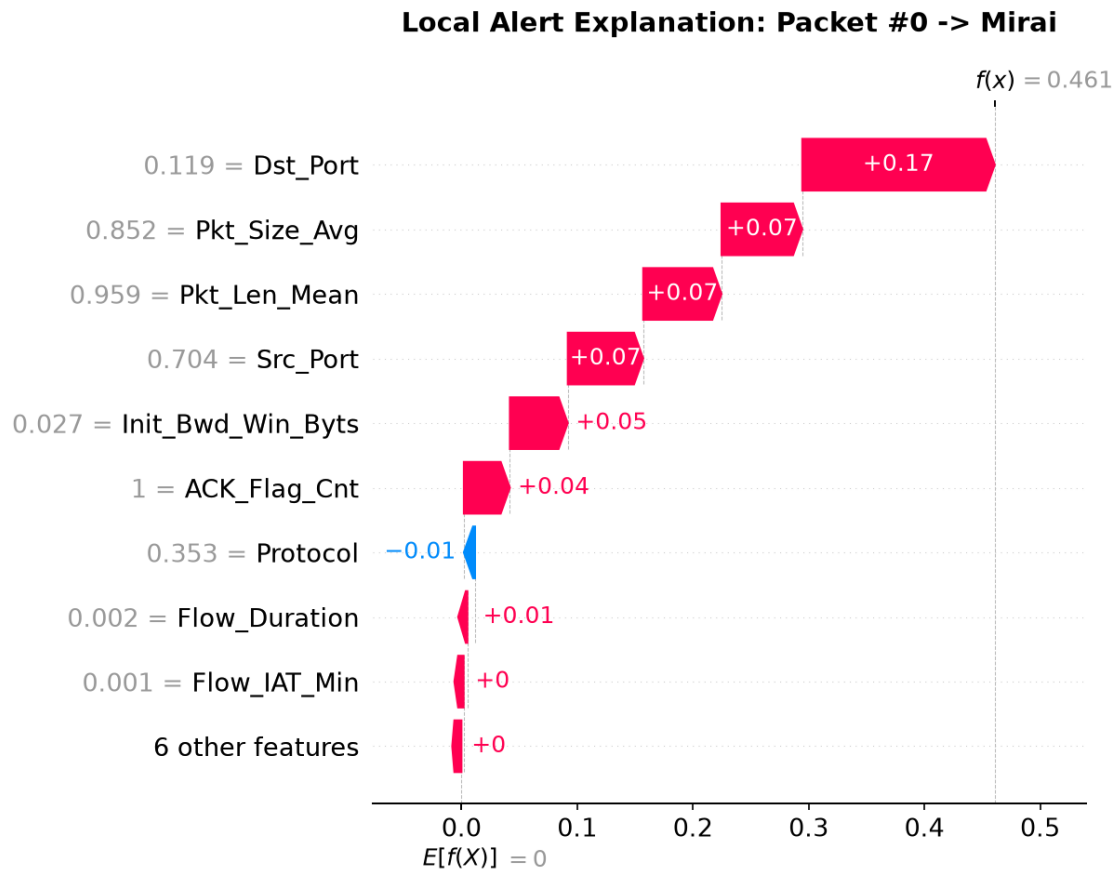


Figure 4.14: SHAP local waterfall chart for a single test sample (index 0), showing the additive contribution of each feature to the model's classification prediction.

In turn, the waterfall chart equips SOC analysts with precise forensic evidence to track down exactly which feature of a packet the model considered anomalous (e.g., an abnormal Src_Port mapping or an irregular ACK_Flag_Cnt). By eliminating the "black-box" nature of deep learning, analysts save hours previously spent manually browsing a plethora of logged information. They can act preemptively to handle localized edge incidents that would have gone unnoticed had the explainable Attention-BiLSTM framework been absent.

4.4 Summary

This chapter analysed the proposed framework within the Three-Tier Success Narrative: **Tier 1** confirmed that the feature reduction with PI+CREA method preserves and enhances classification. Model accuracy, achieving 99.81% binary and 92.62% multi-classification was achieved (1.49 MB model). **Tier 2** exhibited the indefensible collapse in prediction performance of the PGD-10 attacked model to a pathetic 8.21 % without defending. **In Tier 3**, we showed that PGD Adversarial Training is not a silver bullet, it is a deliberate engineering tradeoff: - 4.63 pp drop of classification performance (accuracy: 87.99 %) but robust at 75.36 % to the same PGD-10 attack a gain of 67.15 pp of robustness. Transparency of alerts with global and local feature ranking through SHAP and waterfall explanations respectively, providing undeniable and verifiable threat alerts.

Chapter 5

Engineering Standards and Design Challenges

This chapter lists out the software, hardware & communication standard followed in this project, assesses the societal and environmental impact & maps the research activity mapped against various types of complex engineering problem profiles.

5.1 Compliance with the Standards

5.1.1 Software Standards

Standard Python programming guidelines (PEP8) are followed for developing maintainable software. Keras API operating on TensorFlow 2.10.0 implementing standard neural network layers and graph operations were used to develop deep learning model allowing reproducing the experiments [31]. Explainability layer makes use of the standard SHAP 0.42.1 API for implementing game theory inspired attribution allowing comparison of results.

5.1.2 Hardware Standards

The constrained model architecture is tested on an ARM based IoT edge gateway (Raspberry Pi 4 with Broadcom BCM2711 64-bit ARM Cortex-A72 CPU running at 1.5 GHz and 2 GB RAM) [3]. The model of size 1.49 MB fits perfectly within the storage and RAM size constraints of edge devices (Table5-1 & Table5-2), without requirement for specialist expensive TPU hardware.

5.1.3 Communication Standards

Feature selection and model ingestion is done following the TCP/IP network protocol standards allowing standard network monitoring tools to be used in conjunction with the model (ports, flag sequences, packet sizes and standard TCP/IP headers). The data flow architecture for packet parsing follows the standard framing used for Ethernet (IEEE 802.3) and Wi-Fi(IEEE 802.11 Wi-Fi) frameworks for packet parsing.

5.2 Impact on Society, Environment and Sustainability

5.2.1 Impact on Life

With intrusion detection happening right at the edge of the IoT network, the system protects various IoT based cyber physical critical systems like Smart medical infrastructure, smart home devices & plants etc from being impacted by malicious network attacks to the connected devices securing local user environments from local network intrusion and device exploitation preventing remote access & automated botnet selection.

5.2.2 Impact on Society & Environment

There is a substantial reduction in the requirement to transmit terabytes of network traffic logs to the centralized cloud database as result of edge processing bringing down the communication bandwidth required during feature extraction. This in turn brings down the overall energy footprint of security infrastructure as it removes the requirement to have bigger carbon emission metrics for centralized cloud storage arrays.

5.2.3 Ethical Aspects

The concern resulting from the "black box" of automation where AI takes automatic networking decisions without prior knowledge of the same being used by security analysts is curbed using the SHAP explainability. Security analysts will no longer resort to biased and unverifiable reasons on basis of which certain IPs might go under network blocklist. This in turn will lead to secure, compliant ethical and accurate implementation of routing rules as part of automated anomaly detection.

5.2.4 Sustainability Plan

The modular composition of various sub-models (like feature selection (PI) and classification modules) allows them to be retrained on new network datasets as and when new threat vectors appear. As only retraining is required due to change in threat definitions but new architecture is not needed it removes redesigning the whole architecture again every time. This will also leads to less e waste due in case of hardware for edge gateways with new generations reducing ewaste for replacement gateways resulting in a better green foot print.

5.3 Project Management and Financial Analysis

Agile model of develop over 48 weeks where costs of development primarily involved computing resources and is shown in Table 5-3 above, however, an estimate for commercial implementation is shown below.

Table 5.1: Commercial Deployment Budget Estimate

Item / Resource	Description	Cost (USD)
Edge Hardware	100x Standard IoT Gateways (repurposing existing nodes)	\$0
Compute Resources	AWS GPU compute time for model training and optimization	\$500
Deployment Overhead	Engineering hours for network routing and dashboard integration	\$2,000
Total Cost		\$2,500

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

This qualifies as a 'Complex Engineering Problem as defined by international engineering accreditation (e.g., The Washington Accord). It involves solving non-linear, multi-dimensional security optimization problems, subject to a variety of finite resources. Mapping to key profiles (EP and K) has been provided below:

Depth of Knowledge (EP1, K1, K2, K3): When solving this problem an in-depth engineering knowledge of sequence modeling, game theory and gradient calculations is needed. The custom Attention Bi-LSTM architecture has been mathematically designed to process sequences using Leong style dot product attention while the models weights were hardened to gradient attacks using PGD minimax robust optimization [14]. These concepts belong to advanced machine learning and mathematical statistics, which are beyond foundation engineering principles. Linear algebra is used to select the CREA eigen-space projection matrix W_{pca} (of 3 or 4 components) from the covariance matrix of raw features, while calculus is used to find the first order gradients of the model loss function related to the input vectors to create the PGD-10 adversarial perturbations

Conflicting Requirements (EP2, K5): The project involves a trade-off off-conflicting engineering constraints. The model had to be compact to be able to fit on an edge gateway (less than 2MB), process packets in real-time fashion (less than 0.1ms latency), achieve high clean classification accuracy and be resistant to white box adversarial attacks at the same time. The fact means enhancing one aspect, such as increasing robustness, often comes at the price of reduced performance in another, like standard accuracy. This conflict entails finding the optimum trading off over the Pareto Frontier.

Depth of Analysis (EP3, K8): Due to the high dimensionality of network traffic (69 features) and the non-linear operations of deep neural networks the interactions

between a deep neural network are often non-trivial. Permutation Importance was applied to correctly identifying the feature of most importance for detecting threats with the aid of class weighted surrogate models avoiding exclusion of minority classes followed up with post hoc SHAP analysis, which resulted in the distribution of shapleys values along the length of each sequence.

Familiarity of Issues (EP4, K6): The emergence of threat coming from adversarial evasion represents the next evolution of cyber threats which has only arisen relatively recently. Attackers could dynamically find loss gradients and create entirely never before seen packet perturbations which circumvent previous methods including existing state of the art signature-based firewalls, requiring us to think outside of the box while designing adaptive, robust classes.

Comprehension (K7): This advanced intrusion detection at local edge gateway requires intricate knowledge of system level technical limitations as well as the balance between various trade-offs within the cyber security space. For instance understanding exactly whether the limited computational and memory constraints of edge devices could ever be traded of with strict real-time and Zero-Trust based explainability was of the utmost importance. Understanding these system technical interactions was critical to create a pipeline that could be deployable for edge gateway scenarios.

Interdependence (EP7): The components of the pipeline (data-splits, features selection based on permutation, performing CREA eign-space projection, Attention-BiLSTM classifier and shapley value calculations) are interdependent with a change in sequence size. A Change in percentage used for feature subset could immediately have bearing on model footprint size, parameter-size, latency and level of explanation.

Table 5.1: Mapping with Complex Engineering Problem.

EP1 Dept of Knowled ge	EP2 Range Of Conflicting Requireme nts	EP3 Depth of Analys is	EP4 Familiari ty of Issues	EP5 Extent of Applica ble Codes	EP6 Extent Of Stake- holder Involveme nt	EP7 Interdepende nce
✓	✓	✓	✓	×	×	✓

Mapping with Knowledge Profile

This section is designed to map the overall problem and EP1 (*multiple between K3, K4, K5, K6, K8 for attaining EP1*) to the Knowledge Profile.

Table 5.2: Mapping with knowledge Profile.

K1 Natu ral Scien ce	K2 Mathem atics	K3 Engineeri ng Fundame ntals	K4 Special ist Knowle dge	K5 Enginee ring Design	K6 Enginee ring Practice	K7 Comprehe nsion	K8 Resear ch Literat ure
✓	✓	✓	×	✓	✓	✓	✓

5.4.2 Engineering Activities

Considering the standard activities identified under Washington Accord, the project is mapped to Complex Engineering Activities (EA).

EA1: Range of Resources (EA-1) Integrating multiple software frameworks (TensorFlow, scikit-learn, SHAP) and diverse hardware (NVIDIA GPU acceleration for training, ARM architectures for edge validation).

EA2: Resolution of Interactions (EA-2) Contradiction in preprocessing: If one applies oversampling before traintest splitting, target leakage is caused, but if one applies it after splitting the testing set is alienated, so it requires ratioing with respect of class-weight. To solve this, the sequence of operations was: Split -> Scale -> SMOTE.

EA3: Innovation (EA-3) The innovation of this work is an architectural synthesis instead of using ML libraries-in-Isolation as in previous works. Here we propose a mathematically compressed, adverserially hardened Attention-BiLSTM sequence model for use in resourceful impoverished IoT edge. Instead of using raw, unoptimized data, the pipeline introduces a class-balanced permutation Importance protocol to bypass volumetric spoofing by locking down unto hard routing structures such as ports and protocols. Moreover, it applies a CREA projection to compress the model to 1.49MB (78.3%) but retains original axes for Zero-Trust SHAP explainability. The proposed architecture then applies PGD-10 in adversarial training to Harden the structure turning a historically fragile deep learning architecture into a robust pipelines that can survive whitebox evasion attacks..

EA5: Familiarity of Issues (EA-5) The threat model used- adversarial attacks through evasion is an ever evolving area of study, as the attackers keep on updating the perturbation algorithms used by them.

Mapping with Complex Engineering Activities

This section is designed to map the overall problem and EA's (*multiple*).

Table 5.3: Mapping with Complex Engineering Activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	×	✓

5.5 Summary

This chapter analyzed the proposed framework against global software & hardware standard parameters, checked on the impacts this would have on the environment and ethics and finally mapped out the research to complex engineering problem profile & activities to meet global engineering standards.

Chapter 6

Conclusion

This chapter summarises the contributions made within this research by, discussing through the limitation there by the proposed framework as well as the proposed future work.

6.1 Summary

The purpose built framework presented is in fact an edge-optimized robust and explainable deep learning pipeline for multi-class IoT intrusion detection has three tiers to its engineering achievements that should highlight this thesis success. First Class-Balanced Permutation Importance algorithm and CREA projection from 69 features to 15 features 78.3% reduction for IoTID20 dataset to help compression whilst improving the compression a regular Bi-LSTM performed at 99.64% binary & 89.44% multiclass performance over the standard 69 features for 4.26MB, the edge-optimized Bi-LSTM performed the newly proposed 15 features and provided **99.81% binary plus 0.17 %** for and for **92.80% multi-class performance 3.36 % increased accuracy** 3.63 MB and proposed with an Attention-BiLSTM with **92.62 % multi-class performance** for an ultra-lightweight 1.49 MB device a latency of 0-0368ms was shown Second the analysis of adversarially vulnerability shows that despite the great cleanperformance of the proposed pipeline, it dramatically dropped below **8.21 %** while being attacked with white-box **PGD-10 ($\epsilon=0.05$)**. Third through an ingenious engineering trade-off through applying PGD-10 adversarial training as a measure to counteract this vulnerability at the expense of only decreasing cleanaccuracy to **87.99 % (4.63 % dropped)** under the same attack, increased robustness from 8.21 % to **75.36% which has gained of 67.15** percentage points and finally using with SHAP KernelExplainer, security analysts are provided with global rank list of feature importance as well as local packet level waterfall explanation by this allowing security analysts for alert to be transparent to be acted upon and verifies.

6.2 Limitation

The The proposed framework comes with a main limitation on the accuracy-robustness trade off as incorporating the adversarial training reduced the classification clean accuracy from 92.62 % to 87.99% thus reduction of accuracy by 4.63 % also SHAP kernelExplainer is computationally expensive hence it can not generate local waterfall plot in real time on the actual edge device and therefore should be computed asynchronously a dashboard. Where it has showed exponential

growth in number of features, $\mathcal{O}(2^k \cdot T_f)$; where k is the number of features 15 and T_f is the inference time of the deep neural networks and multi-class classification means these are computed independently for each of the 5 classes hence multiplying it to have several seconds per each packet which is makes real time on the fly explanation in case of packet with edge devices difficult this sort of the explanation are computed in the back-ground (asynchronously) on the centralised security hubs.

6.3 Future Work

In the future, we will: Physical Edge Deployment, Deploy our best model onto physical ARM microcontrollers to investigate the real-world battery consumption and run time performance on board devices. Adaptive Adversarial Training, Test active/dynamic perturbation magnitudes ' ϵ ' during adversarial training to recover clean accuracy while ensuring that adversarial robustness is protected. Threat Mitigation, Connect the explainable dashboard outputs to automated firewall policies to automatically block the threat vector detected and improve responses times even further.

.

References

- [1] I. Ullah and Q. H. Mahmoud, "A Scheme for Generating a Dataset for Anomalous Activity Detection in IoT Networks," in *Advances in Artificial Intelligence, Canadian AI 2020, LNCS*, vol. 12109, Springer, 2020, pp. 508-520.
- [2] S. Akintade, S. Kim, and K. Roy, "Explaining Machine Learning-Based Feature Selection of IDS for IoT and CPS Devices," in *19th IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI)*, Springer, 2023.
- [3] T. B. Ogunseyi and P. Thiagarajan, "An Explainable LSTM-Based Intrusion Detection System Optimized by Firefly Algorithm for IoT Networks," *Sensors*, 2025.
- [4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in NeurIPS*, vol. 30, 2017, pp. 4768-4777.
- [5] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *ICLR*, 2015.
- [6] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards Deep Learning Models Resistant to Adversarial Attacks," in *ICLR*, 2018.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *JAIR*, vol. 16, 2002, pp. 321-357.
- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *KDD '16*, 2016, pp. 785-794.
- [9] M. N. Islam et al., "Towards Machine Learning Based Intrusion Detection in IoT Networks," *CMC*, vol. 69, no. 2, 2021, pp. 1801-1821.
- [10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997, pp. 1735-1780.
- [11] M. Schuster and K. K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Trans. Signal Processing*, vol. 45, no. 11, 1997, pp. 2673-2681.
- [12] X. Chen, M. Liu, Z. Wang, and Y. Wang, "Explainable Deep Learning-Based Feature Selection and Intrusion Detection Method on the Internet of Things," *Sensors*, vol. 24, no. 16, 2024, p. 5223.
- [13] S. Kalakoti, R. Vaarandi, and H. Bahsi, "Evaluating XAI for Deep Learning-Based NIDS Alert Classification," *Sensors*, 2025.
- [14] M.-T. Luong, H. Pham, and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," in *EMNLP*, 2015, pp. 1412-1421.
- [15] H. Ben Ncir, M. Ben HajKacem, and K. A. Alattas, "Enhancing Intrusion Detection Performance Using Explainable Ensemble Deep Learning," *IEEE Access*, 2024.
- [16] A. Grabowski and S. Xu, "Advancing Cybersecurity Practice: Explainable Machine Learning for Network Intrusion Detection," *Journal of Cybersecurity Education, Research and Practice*, 2024.
- [17] A. Al-Shibly, A. Burgas, and J. Massana, "Interpretable Intrusion Detection for IoT: A CNN-BiLSTM Permutation Importance Framework," *Frontiers in Big Data*, 2026.

- [18] A. Abdelaziz et al., "Enhancing Network Threat Detection with Random Forest-Based NIDS and Permutation Feature Importance," *Sensors*, 2025.
- [19] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *KDD '16*, 2016, pp. 1135–1144.
- [20] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron," *IEEE Access*, 2024.
- [21] S. Hariharan, R. R. R. Robinson, R. R. Prasad, C. Thomas, and N. Balakrishnan, "XAI for intrusion detection system: comparing explanations based on global and local scope," *Journal of Computer Virology and Hacking Techniques*, vol. 19, 2023, pp. 217–239.
- [22] C. Zhang, J. Li, N. Wang, and D. Zhang, "Research on Intrusion Detection Method Based on Transformer and CNN-BiLSTM in Internet of Things," *Sensors*, vol. 25, no. 9, 2025, p. 2725.
- [23] S. Kaushik, A. Bhardwaj, A. Almogren, S. Bharany, A. Altameem, A. U. Rehman, S. Hussien, and H. Hamam, "Robust machine learning based Intrusion detection system using simple statistical techniques in feature selection," *Scientific Reports*, vol. 15, 2025, p. 3970.
- [24] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. Springer, 2002.
- [25] S. G. K. Patro and K. K. Sahu, "Normalization: A Preprocessing Stage," *IARJSET*, vol. 2, no. 3, 2015.
- [26] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation," in *IJCAI*, vol. 14, no. 2, 1995, pp. 1137–1145.
- [27] M. Sokolova and G. Lapalme, "A Systematic Analysis of Performance Measures for Classification Tasks," *Information Processing & Management*, vol. 45, no. 4, 2009, pp. 427–437.
- [28] G. Apruzzese et al., "The Role of Machine Learning in Cybersecurity," *Digital Threats*, vol. 4, no. 1, 2023, pp. 1–38.
- [29] N. Capuano et al., "Explainable AI in CyberSecurity: A Survey," *IEEE Access*, vol. 10, 2022, pp. 93575–93600.
- [30] J. Zhang, X. Zhang, Z. Liu, F. Fu, Y. Jiao, and F. Xu, "A Network Intrusion Detection Model Based on BiLSTM with Multi-Head Attention Mechanism," *Electronics*, vol. 12, 2023, p. 4170.
- [31] V. Z. Mohale and I. C. Obagbuwa, "Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability," *Frontiers in Computer Science*, vol. 7, 2025, p. 1520741.
- [32] V. Satopaa et al., "Finding a 'Kneedle' in a Haystack: Detecting Knee Points in System Behavior," in *ICDCSW*, 2011, pp. 166–171.
- [33] M. A. Ferrag et al., "Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study," *J. Info. Sec. Applications*, vol. 50, 2020, p. 102419.
- [34] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, and A. Madry, "Robustness May Be at Odds with Accuracy," in *ICLR*, 2019.
- [35] M. Shafiq et al., "A Hybrid BiLSTM-CNN Approach for Intrusion Detection for IoT Applications," *Electronics*, 2024.
- [36] S. Sadhwani, A. Navare, A. Mohan, R. Muthalagu, and P. M. Pawar, "IoT-

- based Intrusion Detection System Using Explainable Multi-class Deep Learning Approaches," *Computers and Electrical Engineering*, vol. 123, 2025, p. 110256.
- [37] N. G. Pour and S. Shemehsavar, "DCB-VIM: An Ensemble Learning Based Filter Method for Feature Selection with Imbalanced Class Distribution," *Neurocomputing*, vol. 651, 2025, p. 130848.
- [38] D. Alomari, M. A. Alabdullatif, and F. A. Khan, "An Explainable AI-based Network Intrusion Detection System for Botnet Attacks," in *EASE Companion 2025*, 2025, pp. 169-175.
- [39] E. W. Sholeha, D. Y. Jaya, and Q. A. Fitroh, "Explainable Machine Learning for Network Intrusion Detection Using SHAP-Based Feature Interpretation," *Journal of Information Systems and Informatics*, 2024.

Mursaline Huqe

0242220005101804

 Quick Submit Quick Submit Daffodil International University

Document Details

Submission ID

trn:oid::1:3605983038

Submission Date

Jul 4, 2026, 1:17 PM GMT+6

Download Date

Jul 4, 2026, 4:01 PM GMT+6

File Name

FYDP_Final_Report_Mursaline_Huqe_0242220005101804_1.pdf

File Size

2.6 MB

62 Pages**16,454 Words****93,706 Characters**

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **91 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
-  **9 Missing Quotations 1%**
Matches that are still very similar to source material
-  **25 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 12%  Internet sources
- 7%  Publications
- 11%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
692 suspect characters on 10 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 91 Not Cited or Quoted** 12%
Matches with neither in-text citation nor quotation marks
- 9 Missing Quotations** 1%
Matches that are still very similar to source material
- 25 Missing Citation** 2%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 12% Internet sources
- 7% Publications
- 11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers		
	Daffodil International University		5%
2	Internet		
	dspace.daffodilvarsity.edu.bd:8080		2%
3	Student papers		
	Muslim College Multan		1%
4	Internet		
	arxiv.org		<1%
5	Publication		
	"Advances in Artificial Intelligence", Springer Science and Business Media LLC, 2020		<1%
6	Publication		
	C. Manzano, C. Meneses, P. Leger, H. Fukuda. "An Empirical Evaluation of Supervis...		<1%
7	Publication		
	Ashutosh Bhoi, Sthita Pragyan Pujari, Rakesh Chandra Balabantaray. "A deep lear...		<1%
8	Student papers		
	The University of the West of Scotland		<1%
9	Internet		
	www.internationalphoneticassociation.org		<1%
10	Publication		
	Ibrahim Al-Shibly, Llorenç Burgas, Joaquim Massana. "Interpretable intrusion det...		<1%

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.